

POST AND TELECOMMUNICATION INSTITUTE OF TECHNOLOGY

FACULTY OF INFORMATION TECHNOLOGY 1



INFORMATION SYSTEM ANALYSIS AND DESIGN

Class: E22HTTT

Subject No.28 A garage management system

Fullname: Nguyễn Phi Nhật

Student_id B22DCCN580

TABLE OF CONTENT

	1
Subject No. 28	3
I. CONCEPT EXPLORATION	4
Build the Glossary list	4
II. BUSINESS MODEL	8
A. Requirements:	8
A.1. By Natural Language:	8
1. Objective	8
2. Scope	8
3. How do modules work	9
4. Information about objects	11
5. Relationships among objects	11
A.2. By UML	12
1. Use case diagram (for the whole system)	12
2. Use case diagram (Customer searches for service information)	13
3. Use case diagram (Sale staff receives payment)	14
B. Analysis:	14
1. Scenario	14
a. Module: Customer searches for service information	14
b. Module: Sale staff receives payment	17
2. Extracting Entity Classes	20
3. Classes diagram and state diagram	24
Classes diagram for Search for Service Information	24
Classes diagram for Search for Spare parts Information	24
Classes diagram for Receive payment	25
State Diagram for Search for Service Information	26
State Diagram for Search for Spare parts Information	27
State Diagram for Receive payment	28
4. Communication diagram and sequence diagram	28
Scenario 2 for searching for service information	28
Communication Diagram for Search Service Information	30

Sequence diagram for Search for Service Information	31
Sequence diagram for Search for Spare parts Information	31
Scenario 2 for Receive payment	31
Communication Diagram for Receive Payment	32
Sequence diagram for Receiving payment	33
C. Design	33
1. Entity class diagram	33
2. Database Design	34
3. Class diagram for Search for Service information	36
4. Class diagram for Receive Payment	37
5. Activity for Search for Service information	37
5.1. Service	37
5.2. SpareParts	38
6. Scenario v3 for Search for Service information	38
7. Sequence for Search for Service information	41
8. Scenario v3 for Receive Payment	42
9. Sequence for Search for Receive Payment	45
10. Package diagram	46
11. Deployment diagram	47
D. CODE	48
1. File Structure	48
2. Interface	50
2.1. Module 1:	50
Welcome.jsp	51
CustomerHomeView.jsp	51
SearchForServiceInformationView.jsp	51
ServiceDetailsView.jsp	52
SearchForSparePartsInformationView.jsp	52
SparePartsView.jsp	53
2.2 Module 2	54
Welcome.jsp	54
HomeViewSaleStaff.jsp	54
SelectCustomerView.jsp	55
DetailedInvoicesView.jsp	56
PaymentView.jsp	57
3. GITHUB LINK	57

Subject No. 28

Duration: 90 minutes

A garage management system enables management staff, sales staff, warehouse staff, and customers to use:

- Management staff: view the statistics categories: services, parts, customers and suppliers. Service information management, spare parts.
- Warehouse staff: import spare parts from suppliers, manage supplier information
- Sales staff: receive customers, receive service and spare parts requests from customers, assign technical staff according to customer service requirements, receive payment from customers.
- Customer: Search service information, book an appointment online

Considering two modules and answering questions

- ***Customer searches for service information:*** selects the menu to find service / spare parts information → enters the service / part name to search → the system displays a list of services / parts whose name contains the entered keyword → clicks on a service/part to view details → the system displays detailed information about the service/part.
- ***Sales staff receives payment:*** selects the payment menu → finds and selects customers by name → view detailed invoices containing a list of used spare parts and services (can be added if needed) → invoice confirms → Prints invoices and receives payments from customers.

Question 1 (2 points)

- a. Build the use case diagram for the two modules
- b. Present scenarios for the two modules

Question 2 (2 points)

- a. Extract the entity classes (class name, attributes) related to the two modules
- b. Build the entity class diagram for the extracted classes.

Question 3 (2 points)

- a. Build the communication diagram for the standard scenario of the two modules.
- b. Build the detail design class diagram for the two modules.

Question 4 (2 points)

- a. Based on the entity classes of Question 2, design the database related to the two modules
- b. Generate Java code (class template, declaration of attributes and methods, explain how methods work) for classes in the diagram of Question 3.b.

Question 5 (2 points)

- a. Build the package diagram for classes in the diagram of Question 3.b.
- b. Build the deployment diagram for MVC model based on the J2EE technology.

I. CONCEPT EXPLORATION

Build the Glossary list

No.	English concept	Meaning	Category
-----	-----------------	---------	----------

1	Management staff	The person responsible for supervising the entire garage operation (services, warehouse, sales, customer care). They monitor KPIs such as service/parts revenue, technician productivity, customer retention, and outstanding payments. They have the authority to approve price lists and discounts, view consolidated reports, and configure system parameters.	Actor
2	Customer	An individual or organization that uses the garage's services. They can search for services and spare parts, create appointments, receive quotations, make payments, and obtain invoices. A customer profile contains contact details, vehicle information, service history, warranty records, and special notes (e.g., specific repair requests).	Actor
3	Technical staff	Personnel who directly perform maintenance and repair tasks. They receive assignments from sales staff or coordinators, update progress, and record parts consumed in each job. They also perform quality checks after repair and suggest additional replacements if required.	Actor
4	Sales staff	The staff member who interacts with customers, records service/part requests, assigns technicians, generates and confirms invoices, prints invoices, and processes payments. They are the main communication channel with customers throughout the workflow and may apply approved discounts or promotions.	Actor
5	Warehouse staff	Responsible for managing the stock of spare parts and materials. They create purchase entries from suppliers, issue parts according to work orders, perform periodic inventory checks, and monitor expiry dates or serial numbers. The system should track minimum stock levels, provide low-stock alerts, and maintain lot history.	Actor
6	Service	A package of tasks provided by the garage, such as periodic maintenance, oil changes, brake repair, tire balancing, or electrical diagnostics. Each service has a code, description, standard duration, labor cost,	Entity

		recommended parts/consumables, and a quality checklist.	
7	Spare parts	Parts and consumables used in vehicle repair, such as filters, oil, brake pads, or spark plugs. Each item is managed by code, name, supplier, stock quantity, purchase price, selling price, storage location, and unit of measure. Optional tracking may include lot/serial numbers and expiration dates. Spare parts are linked to services so the system can auto-suggest required materials.	Entity
8	Invoice	The official document recording payment for services and spare parts provided to a customer. It includes line items, quantities, unit prices, taxes/discounts, totals, status (draft/confirmed/paid), payment method, and details for printing or archiving.	Entity
9	Appointment	A scheduled booking created by a customer for a specific date and time, linked to a chosen service and vehicle. It helps allocate workshop resources (bays, lifts, technicians). Appointment statuses include: created, confirmed, late arrival, completed, or canceled. Notifications can be sent by SMS/email.	Entity
10	Supplier	An external business entity that provides spare parts, components, or materials to the garage. Suppliers interact with the system indirectly through warehouse staff, who create purchase entries and record delivery information. In this context, the supplier is not managed as an internal user but as an external actor that supplies goods. The system needs supplier details (name, contact, payment terms, reliability) in order to support procurement, inventory updates, and financial reconciliation.	Actor
11	Car	The customer's vehicle information, including license plate, make, model, year, VIN, mileage, and service/repair history. The system uses this data to suggest appropriate service packages and compatible spare parts.	Entity

12	Import spare parts	The process performed by Warehouse staff to bring new spare parts into the garage's inventory from external suppliers. It includes recording supplier details, entering spare part names, quantities, unit prices (if available), and updating the stock database. This function ensures that the spare parts needed for repair and maintenance services are available in the warehouse. It may also include verifying delivery documents, checking part quality, and confirming the receipt of goods.	Process
13	View Statistics	A management function that provides reports and analytics: revenue by day/month/service, profit per parts group, service frequency, technician productivity, stock turnover, top customers, and promotion effectiveness. Reports can be filtered by time range/branch and exported (PDF/Excel).	Process
14	Manage Information	The CRUD function for maintaining information about services, spare parts, and suppliers (optionally customers and vehicles). Includes price settings, service-part relationships, tax/discount policies, stock minimums, and role-based access control.	Process
15	Select service technician	A function used by sales staff when receiving a car for repair. After identifying the customer and listing the required services and spare parts, the sales staff chooses an available technical staff member to perform the work. This step ensures that each repair or maintenance task is assigned to a specific technician, based on expertise, availability, and workload. Once confirmed, the assignment is saved into the system, becoming part of the work order for tracking progress and accountability.	Process
16	Payment	The process of collecting money from customers. It involves confirming the invoice, selecting a payment method (cash, card, QR, bank transfer), recording full or partial amounts, printing the receipt, and marking the invoice as paid. The system stores transaction codes, operator details, timestamps, and updates accounts or	Process

		loyalty points. Refunds and adjustments are also supported according to policy.	
17	Manage supplier information	A function used mainly by warehouse staff to record, update, and maintain all relevant data about suppliers. This includes the supplier's name, contact details, address, payment terms, product categories, delivery schedules, and reliability rating. Through this function, staff can add new suppliers, edit existing records, or deactivate suppliers no longer in use. Proper management of supplier information ensures accurate tracking of spare part sources, supports purchasing decisions, and facilitates auditing and reporting on supplier performance.	Process

II. BUSINESS MODEL

A. Requirements:

A.1. By Natural Language:

1. Objective

The GaraMan system is developed to manage all activities of the automobile garage, including the management of services, spare parts, customers, suppliers, and payment operations. The main goal is to optimize workflows, improve management efficiency, and provide customers with a convenient experience when using the garage's services.

2. Scope

The GaraMan system is used by four main user groups:

- Management Staff: Responsible for overall management, monitoring, and evaluating activities related to services, spare parts, customers, and suppliers.
- Warehouse Staff: Responsible for importing, exporting, and managing spare parts from suppliers.

- **Sales Staff:** Responsible for receiving customers, recording service and spare part requests, assigning tasks to technical staff, and handling payments.
- **Customers:** End users of the services, who can search for service and spare part information and book appointments online.

3. How do modules work

- **Service/Spare Part Information Search:** Select the “Search Service/Spare Part Information” menu → enter a keyword (service or part name) into the search box → the system searches and displays a list of services/parts whose names contain the entered keyword → the customer clicks on a service/part from the list → the system displays detailed information about the service/part (including description, price, execution time, or supplier information).
- **Spare Part Entry Function (Warehouse Staff):** Select the “Enter Spare Parts” menu → search and select a supplier by name (if not available, add a new supplier with information such as name, address, phone number) → repeat until all spare parts are entered: search and select a spare part by name (if not available, add a new spare part with information such as name, part code, description) → enter the quantity and unit price of each spare part → when finished entering all items, confirm the invoice → the system prints the spare part purchase invoice → proceed with payment and save the invoice information for the supplier.
- **Online Appointment Booking (Customer):** Select the “Book Appointment” menu → choose a suitable date and time → enter the customer’s personal information (including name, phone number, license plate number, and additional details if needed) → confirm the booking → the system notifies success and displays the details of the booked appointment.
- **Vehicle Reception Function (Sales Staff):** Select the “Receive Vehicle” menu → search and select the customer by name (add a new customer if not available, with information such as name, phone number, and license plate number) → repeat until all required services and spare parts are entered: search and select the service/part by name (add new if not available) → when finished, assign a technical staff member to perform

the service → confirm the reception information → the system notifies success and records the information into the database.

- **Payment Processing Function (Sales Staff):** Select the “Payment” menu → search and select the customer by name (or by other information such as license plate number) → view the detailed invoice, including the list of spare parts and services used by the customer → add services/parts if necessary → confirm the invoice → print the invoice → receive payment from the customer and save the transaction information.
- **Online Appointment Booking (Customer – simplified):** Select the “Book Appointment” menu → choose the desired date and time → enter the customer’s personal information (name, phone number, license plate number, special requests) → confirm the booking → the system notifies success.
- **Service/Spare Part Revenue Report (Management Staff):** Select the “View Report” menu → choose the report type: services or spare parts by revenue → select the start and end dates of the reporting period → view the list of services/parts along with their corresponding total revenue → click on a service/part to view details → see the list of times the service/part was used within the selected period → click on a usage record to view the corresponding detailed invoice.
- **Customer Revenue Report (Management Staff):** Select the “View Report” menu → choose the report type “Customers by Revenue” → select the start and end dates of the report → view the list of customers along with the total revenue from each → click on a customer to view details → see the list of service visits by that customer within the selected period → click on a visit to view the corresponding detailed invoice.
- **Supplier Report by Quantity of Goods Imported (Management Staff):** Select the “View Report” menu → choose the report type “Suppliers by Quantity Imported” → select the start and end dates of the report → view the list of suppliers along with the total quantity of goods imported from each → click on a supplier to view details → see the list of spare part imports from that supplier within the selected period → click on an import record to view the corresponding detailed purchase invoice.

4. Information about objects

System Users: account, password, full name, address, date of birth, email, phone number, gender, role.

Staff: same as user, plus position.

Management Staff: same as staff.

Sales Staff: same as staff.

Warehouse Staff: same as staff.

Customer: same as user.

Service: name, price, description.

Spare Part: name, price, description, remaining quantity.

Invoice: time, total amount, status, sales staff, customer, vehicle, list of services, spare parts.

Appointment: creation time, appointment time, status, customer.

Supplier: name, address, phone number.

Vehicle: description, license plate number, customer.

Import Invoice: time, total amount, status, warehouse staff, supplier, list of spare parts.

Reports:

- Customer revenue report.
- Service revenue report.
- Spare part revenue report.
- Supplier report by quantity of goods imported.

5. Relationships among objects

A customer can have many appointments.

A customer can have many vehicles.

A payment invoice includes information about one sales staff member.

A payment invoice includes information about one customer.

A payment invoice includes information about one vehicle.

A payment invoice can include many spare parts, and one spare part can appear in many payment invoices.

A payment invoice can include many services, and one service can appear in many payment invoices.

An import invoice includes one warehouse staff member.

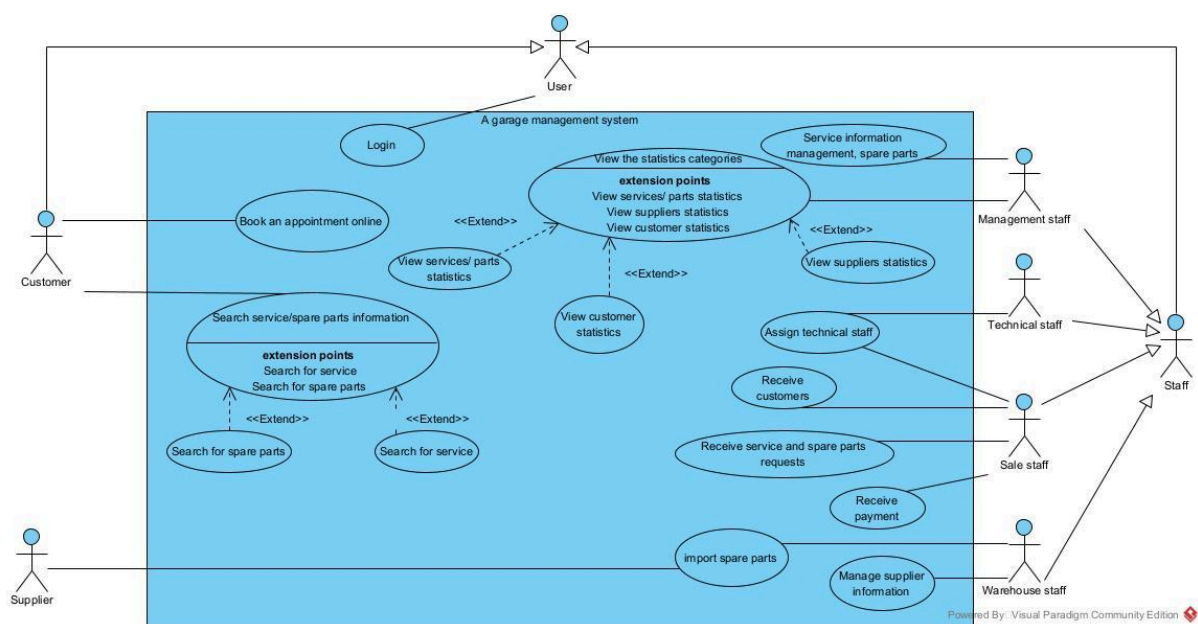
An import invoice includes one supplier.

An import invoice can include many spare parts, and one spare part can appear in many import invoices.

An import invoice can include many services, and one service can appear in many import invoices.

A.2. By UML

1. Use case diagram (for the whole system)

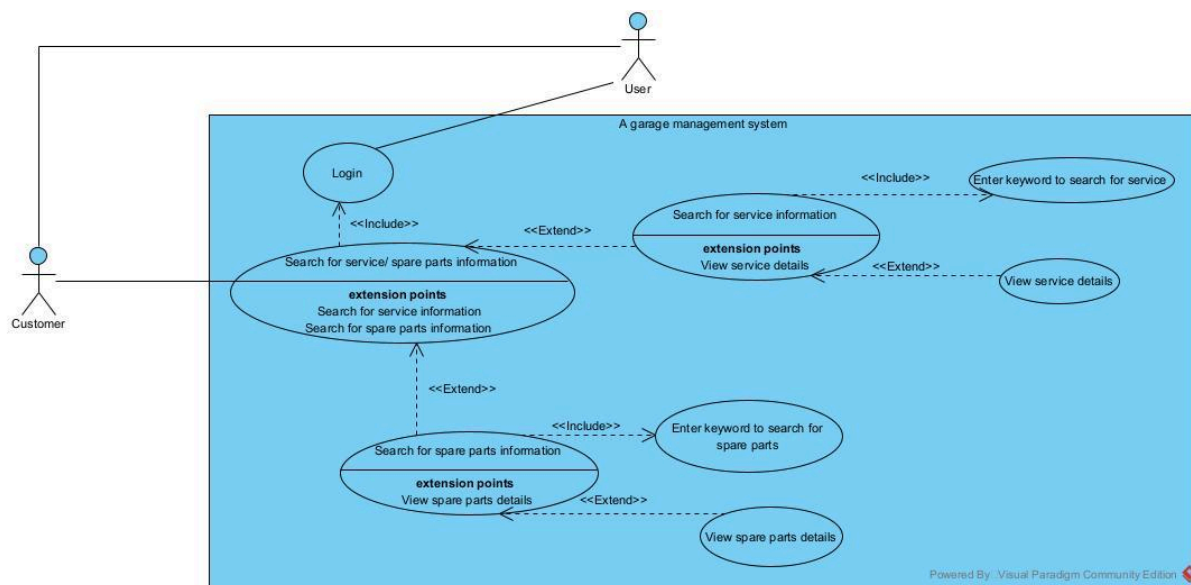


Overview of Use Cases

- **Search Services/Spare Parts:** Allows customers to search for and view detailed information about available services or spare parts in the system based on keywords.
- **Enter Spare Parts:** Allows warehouse staff to add new spare parts from suppliers and manage quantities and pricing.

- **Book Appointments Online:** Allows customers to choose a date/time and book service appointments at the garage through the system.
- **Receive Vehicle for Repair:** Allows staff to receive vehicles from customers, select required services and spare parts, and assign technicians to perform repairs.
- **Payment:** Allows staff to view a customer's detailed invoice and process payment after services are completed.
- **Service/Spare Part Revenue Report:** Allows staff to view revenue statistics for services or spare parts within a specified period.
- **Customer Revenue Report:** Allows staff to view revenue statistics by customer and see details of the service invoices used by each customer.

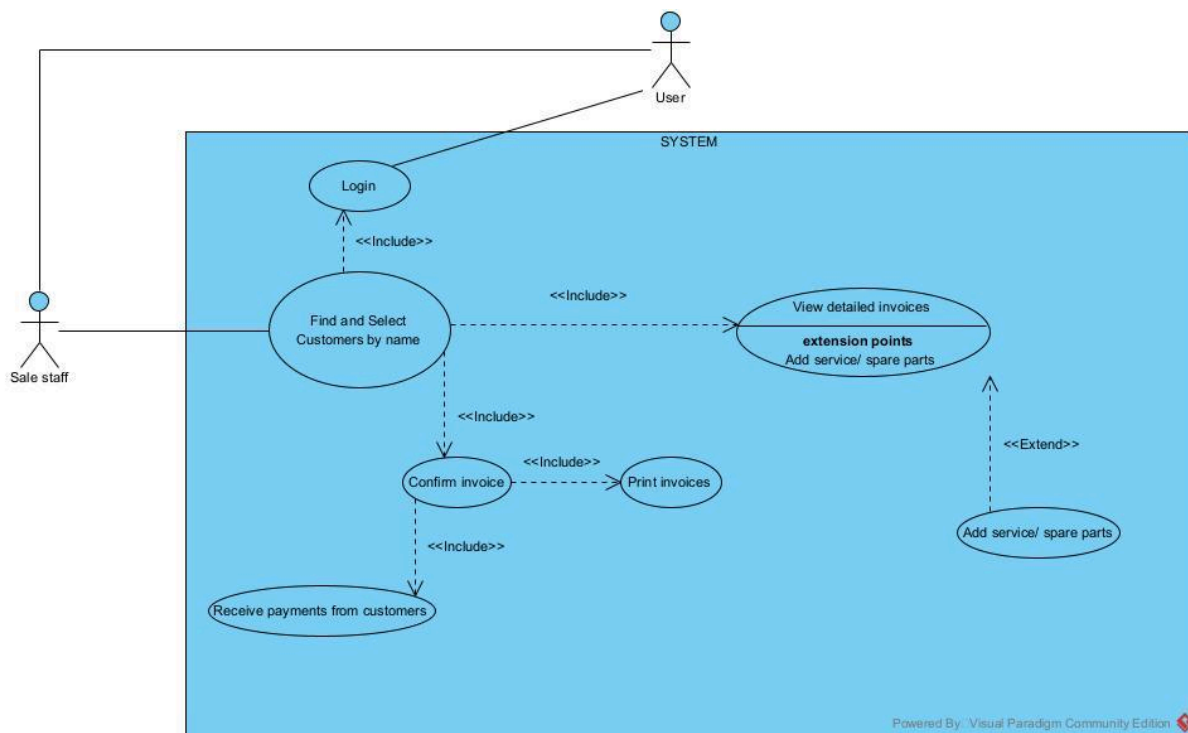
2. Use case diagram (Customer searches for service information)



Use Case Description: Search Service/Spare Part Information

- Select the “Search Service/Spare Part Information” menu → triggers the UC: Search Service/Spare Part Information.
- Enter the service/part name to search → the system displays a list of services/parts matching the keyword.
- Click on a service/part to view details → triggers the UC: View Service/Spare Part Details.
- The service/part detail interface extends from the service/part list interface.

3. Use case diagram (Sale staff receives payment)



Use Case Description: Sales Staff – Process Payment

- Select the “Payment” menu → triggers UC: Search and Select Customer by Name.
- Enter the customer’s name to search → the system displays a list of customers matching the keyword.
- Select a customer → triggers UC: View Invoice Details.
- The invoice detail interface extends from the customer list, displaying the services and spare parts used.
- *(Optional)* Add new services/spare parts → triggers UC: Add Service/Spare Part.
- Confirm the invoice → triggers UC: Print Invoice.
- Once the invoice is successfully printed → proceed with receiving payment from the customer and complete the process.

B. Analysis:

1. Scenario

a. Module: *Customer searches for service information*

Use case	Search for Service Information
Actor	Customer

Preconditions	The customer has successfully logged in. The system already contains service information.															
Postconditions	The customer finds services whose names contain the entered keyword.															
Main Scenario	1.The customer selects the “Search for Service Information” menu from the main screen after successful login. 2.The search interface appears. 3. The customer clicks the input box and enters a keyword, for example “GLC300.” 4. The system displays a list of services whose names contain the entered keyword. <table><tr><td>No.</td><td>Name</td><td>Price</td></tr><tr><td>1</td><td>Mercedes GLC300 Engine Maintenance</td><td>1,000,000 VND</td></tr><tr><td>2</td><td>Car Odor Removal and Bacteria/Mold Elimination for GLC300</td><td>2,000,000 VND</td></tr></table> 5. The customer clicks on “Mercedes GLC300 Engine Maintenance.” 6. The detailed information interface appears: <table><tr><td>Name</td><td>Mercedes GLC 300</td></tr><tr><td>Price</td><td>1,000,000 VND</td></tr></table>			No.	Name	Price	1	Mercedes GLC300 Engine Maintenance	1,000,000 VND	2	Car Odor Removal and Bacteria/Mold Elimination for GLC300	2,000,000 VND	Name	Mercedes GLC 300	Price	1,000,000 VND
No.	Name	Price														
1	Mercedes GLC300 Engine Maintenance	1,000,000 VND														
2	Car Odor Removal and Bacteria/Mold Elimination for GLC300	2,000,000 VND														
Name	Mercedes GLC 300															
Price	1,000,000 VND															

	<table><tr><td>Description</td><td>The Mercedes GLC 300 engine maintenance service includes inspection and maintenance of key engine components to ensure stable and efficient operation of the vehicle. Regular maintenance helps detect potential issues early, replace engine oil, check filters, the cooling system, and related parts. This extends the engine's lifespan and improves vehicle performance. The service gives customers peace of mind about their vehicle's performance while reducing long-term repair costs.</td></tr><tr><td colspan="2"></td></tr></table>	Description	The Mercedes GLC 300 engine maintenance service includes inspection and maintenance of key engine components to ensure stable and efficient operation of the vehicle. Regular maintenance helps detect potential issues early, replace engine oil, check filters, the cooling system, and related parts. This extends the engine's lifespan and improves vehicle performance. The service gives customers peace of mind about their vehicle's performance while reducing long-term repair costs.		
Description	The Mercedes GLC 300 engine maintenance service includes inspection and maintenance of key engine components to ensure stable and efficient operation of the vehicle. Regular maintenance helps detect potential issues early, replace engine oil, check filters, the cooling system, and related parts. This extends the engine's lifespan and improves vehicle performance. The service gives customers peace of mind about their vehicle's performance while reducing long-term repair costs.				
Exceptions	3. If no service exists with a name containing the entered keyword: ● The customer may re-enter another keyword to continue searching.				

Use Case	Search for Spare Parts Information
Actor	Customer
Preconditions	The customer has successfully logged in. The system already contains spare parts information.
Postconditions	The customer finds spare parts whose names contain the entered keyword.
Main Scenario	1.The customer selects the “Search for Spare Parts Information” menu from the main screen after successful login. 2.The search interface appears. 3. The customer clicks the input box and enters a keyword, for example “GLC300.” 4.The system displays a list of services/spare parts whose names contain the entered keyword.

	<table><tr><th>No.</th><th>Name</th><th>Stock Quantity</th><th>Price</th></tr><tr><td></td><td>Genuine Denso Engine Air Filter for Mercedes GLC300 2022-2024</td><td>10</td><td>1,000,000 VND</td></tr><tr><td></td><td>GLC300 Headlight</td><td>20</td><td>2,000,000 VND</td></tr></table> 5.The customer clicks on “Genuine Denso Engine Air Filter for Mercedes GLC300 2022-2024.” 6.The detailed information interface appears: <table><tr><td>Name</td><td>Genuine Denso Engine Air Filter for Mercedes GLC300 2022-2024</td></tr><tr><td>Price</td><td>1,000,000 VND</td></tr><tr><td>Stock Quantity</td><td>10</td></tr><tr><td>Description</td><td>The car deodorizer eliminates harmful mold and bacteria inside the vehicle, purifies the air, removes odors, and sterilizes the car interior. It has a compact size, is easy to use, and can be applied to many different car models.</td></tr></table>	No.	Name	Stock Quantity	Price		Genuine Denso Engine Air Filter for Mercedes GLC300 2022-2024	10	1,000,000 VND		GLC300 Headlight	20	2,000,000 VND	Name	Genuine Denso Engine Air Filter for Mercedes GLC300 2022-2024	Price	1,000,000 VND	Stock Quantity	10	Description	The car deodorizer eliminates harmful mold and bacteria inside the vehicle, purifies the air, removes odors, and sterilizes the car interior. It has a compact size, is easy to use, and can be applied to many different car models.
No.	Name	Stock Quantity	Price																		
	Genuine Denso Engine Air Filter for Mercedes GLC300 2022-2024	10	1,000,000 VND																		
	GLC300 Headlight	20	2,000,000 VND																		
Name	Genuine Denso Engine Air Filter for Mercedes GLC300 2022-2024																				
Price	1,000,000 VND																				
Stock Quantity	10																				
Description	The car deodorizer eliminates harmful mold and bacteria inside the vehicle, purifies the air, removes odors, and sterilizes the car interior. It has a compact size, is easy to use, and can be applied to many different car models.																				
Exceptions	3. If no spare parts exist with names containing the entered keyword: ● The customer may re-enter another keyword to continue searching.																				

b. Module: Sale staff receives payment

Use case	Sale Staff Receives Payment
Actor	Sale staff
Preconditions	Sale staff has successfully logged in.

	The system already contains customer and invoice information.			
Postconditions	Payment is recorded successfully in the system.			
Main Scenario	1.The sale staff selects the “Payment” menu from the main screen after successful login.			
	2.The search interface appears.			
	3.The sale staff finds and selects a customer by name, for example “Nguyen Van A.”			
	4.The system displays detailed invoices of Nguyen Van A containing a list of used services and spare			
	Invoice No.		Date	Status
	INV-2025-001		14/09/2025	Unpaid
	When the sale staff opens the invoice INV-2025-001, the system shows:			
	No.	Item	Quantity	Price
	1	Engine Oil Replacement (Mercedes GLC300)	1	1,000,000
	2	Genuine Air Filter (Mercedes GLC300 2022-2024)	1	2,000,000
Total:3,000,000				
5.(Optional) The sale staff can add more services/spare parts before confirming.				
+ Example: Nguyen Van A also requests Car Odor Removal → the staff adds:				
No.	Item	Quantity	Price	
3	Car Odor Removal and Bacteria/Mold Elimination for GLC300	1	2,000,000	
New total: 5,000,000				
6.The sale staff confirms the invoice.				
Invoice Information (before payment)				
● Invoice No.: INV-2025-001				
● Customer: Nguyen Van A — Vehicle: Mercedes GLC300				

- Date 14/09/2025
- Cashier: Sale staff #SS01
- Status: Confirmed (Locked)

7.The sale staff prints the invoice.

Invoice Details

No.	Item	Quantity	Price
1	Engine Oil Replacement (Mercedes GLC300)	1	1,000,000
2	Genuine Air Filter (Mercedes GLC300 2022-2024)	1	2,000,000
3	Car Odor Removal and Bacteria/Mold Elimination for GLC300	1	2,000,000
Total price			5,000,000

8.The sale staff receives payment from Nguyen Van A.

9.The system updates the invoice status to *Paid*.

Invoice Information (before payment)

- Invoice No.: INV-2025-001
- Customer: Nguyen Van A — Vehicle: Mercedes GLC300
- Date 14/09/2025
- Cashier: Sale staff #SS01
- Status: Paid (Locked)

Exceptions

2. Customer not found

- If the system cannot find “Nguyen Van A” or any matching customer, it shows: *“No customer found.”*
- The sale staff may re-enter the search keyword or select another customer.

3. No unpaid invoices

- If the selected customer has no unpaid invoices, the system displays: *“No unpaid invoices available.”*
- The sale staff may return to the search interface or check invoice

	history. 4. Adding unavailable item ● If the sale staff tries to add a spare part or service that is out of stock or not in the catalog, the system shows: <i>“Item not available.”</i> ● The staff can adjust the quantity or choose another item.
--	--

2. Extracting Entity Classes

Step 1: Describe the whole system in a complete paragraph

The garage management system is a comprehensive software solution designed to streamline the operations of a vehicle repair shop, serving multiple user roles. Management staff utilize the system to view statistics categories including services, parts, customers, and suppliers, alongside managing service and spare parts information. Warehouse staff are responsible for importing spare parts from suppliers and managing supplier information. Sales staff manage the customer workflow: receiving customers, receiving service and spare parts requests, assigning technical staff according to customer service requirements, and finally, receiving payment from customers. Customers can use the system to search for service information and book appointments online. The system also supports key processes like: Customer searches for service information—where the user selects the menu to find service/spare parts information, enters the service/part name to search, the system displays a list of services/parts whose name contains the entered keyword, the user clicks on a service/part to view details, and the system displays detailed information about the service/part. Additionally, the Sales staff receives payment module involves the staff selecting the payment menu, finding and selecting customers by name, viewing detailed invoices containing a list of used spare parts and services (which can be added if needed), confirming the invoice, printing the invoices, and receiving payments from customers.

Step 2: Extract nouns appearing in the topic (not necessarily in the paragraph)

- Nouns related to people: management staff, warehouse staff, sales staff, technical staff, customer.

- Nouns related to information: supplier, service, spare part, payment invoice, purchase invoice, appointment, vehicle, spare parts statistics, service statistics, customer statistics, supplier statistics.

Step 3: Evaluate and select nouns as entity classes or attributes

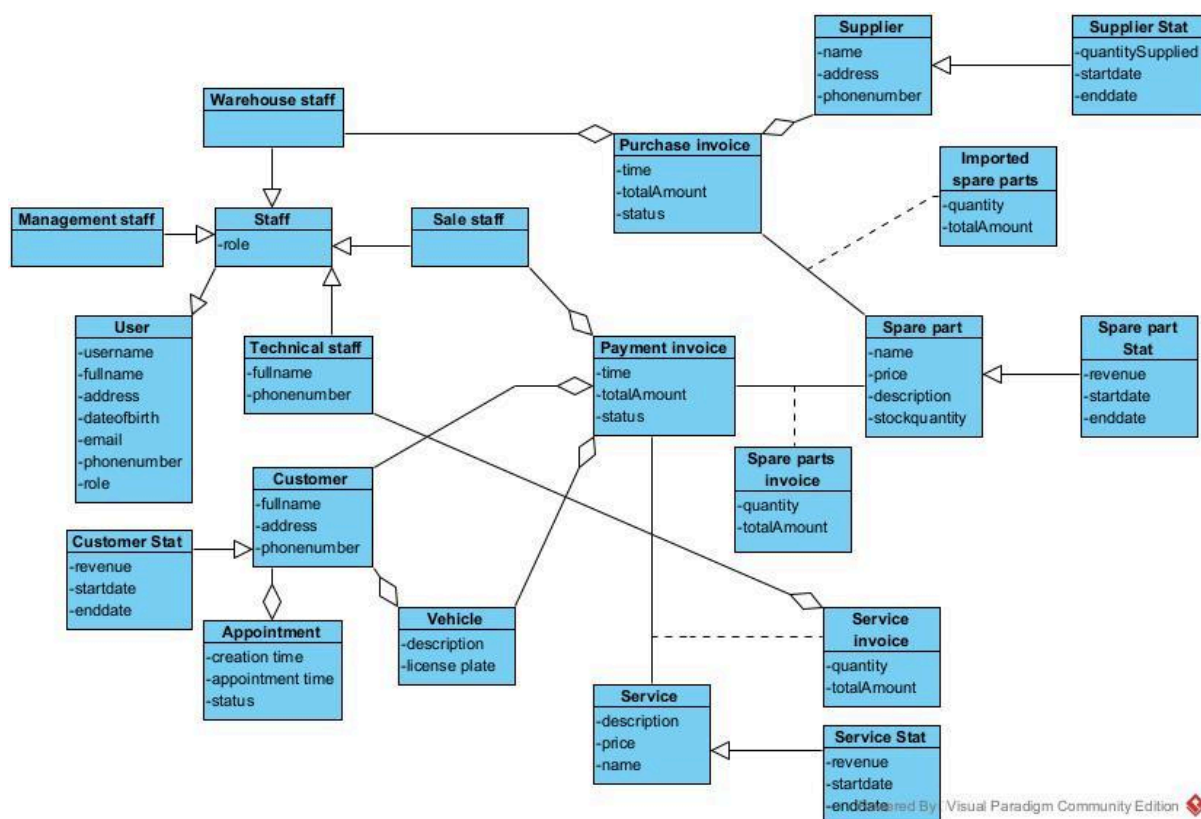
- Nouns related to people:
 - System user → class *User* (abstract): full name, username, password, date of birth, address, email, phone number, role.
 - Staff → class *Staff*: role.
 - Management staff → class *Management staff*.
 - Customer → class *Customer*.
 - Warehouse staff → class *Warehouse staff*.
 - Sales staff → class *Sales staff*.
 - Technical staff → class *Technical staff*.
 - *Staff* and *Customer* inherit from *User*.
 - *Warehouse staff*, *Sales staff*, *Management staff*, *Technical staff* inherit from *Staff*.
- Nouns related to information:
 - Service → class *Service*: name, price, description.
 - Spare part → class *Spare part*: name, price, description, stock quantity.
 - Vehicle → class *Vehicle*: description, license plate.
 - Appointment → class *Appointment*: creation time, appointment time, status.
 - Supplier → class *Supplier*: name, address, phone number.

- Payment invoice → class *Payment invoice*: time, total amount, status.
- Purchase invoice → class *Purchase invoice*: time, total amount, status.
- Statistics classes:
 - Spare parts statistics → class *Spare parts Stat*: revenue, start date, end date.
 - Service statistics → class *Service Stat*: revenue, start date, end date.
 - Customer statistics → class *Customer Stat*: revenue, frequency, start date, end date.
 - Supplier statistics → class *Supplier Stat*: quantity supplied, start date, end date.

Step 4: Relationships between entity classes

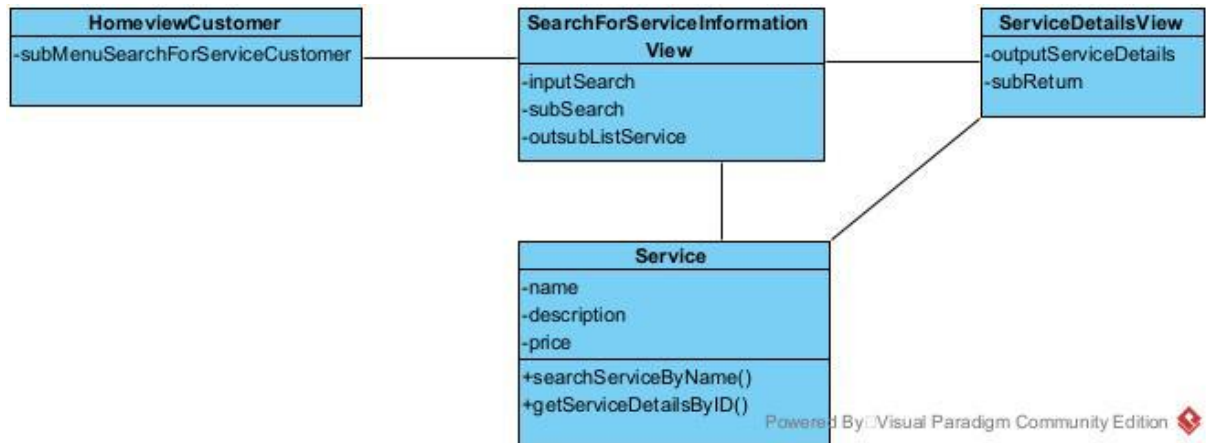
- One customer can have many appointments → the relationship between *Customer* and *Appointment* is 1–n.
- One customer can own many vehicles → the relationship between *Customer* and *Vehicle* is 1–n.
- One sales staff can create many payment invoices → the relationship between *Sale staff* and *Payment invoice* is 1–n.
- One customer can have many payment invoices → the relationship between *Customer* and *Payment invoice* is 1–n.
- One vehicle can belong to many payment invoices → the relationship between *Vehicle* and *Payment invoice* is 1–n.
- One payment invoice can include many spare parts, and one spare part can belong to many invoices → relationship between *Payment invoice* and *Spare part* is n–n → propose class *Spare parts invoice* to represent a spare part in a payment invoice.

- One payment invoice can include many services, and one service can belong to many invoices → relationship between *Payment invoice* and *Service* is n–n → propose class *Service invoice* to represent a service in a payment invoice.
- One technical staff can be assigned to many service invoices → relationship between *Technical staff* and *Service invoice* is 1–n.
- One purchase invoice can include many spare parts, and one spare part can belong to many purchase invoices → relationship between *Purchase invoice* and *Spare part* is n–n → propose class *Imported spare parts* to represent a spare part in a purchase invoice.
- One supplier can have many purchase invoices → the relationship between *Supplier* and *Purchase invoice* is 1–n.
- One warehouse staff can have many purchase invoices → the relationship between *Warehouse staff* and *Purchase invoice* is 1–n.

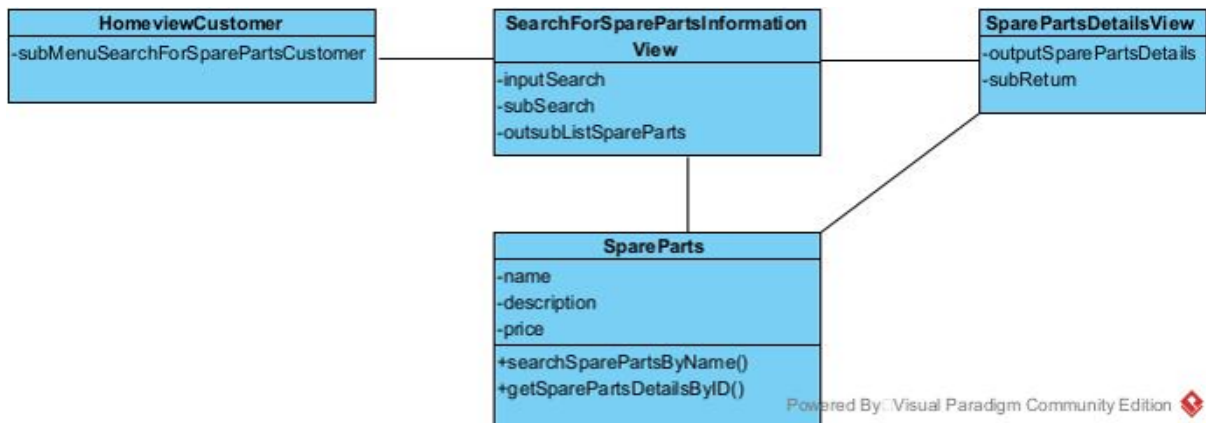


3. Classes diagram and state diagram

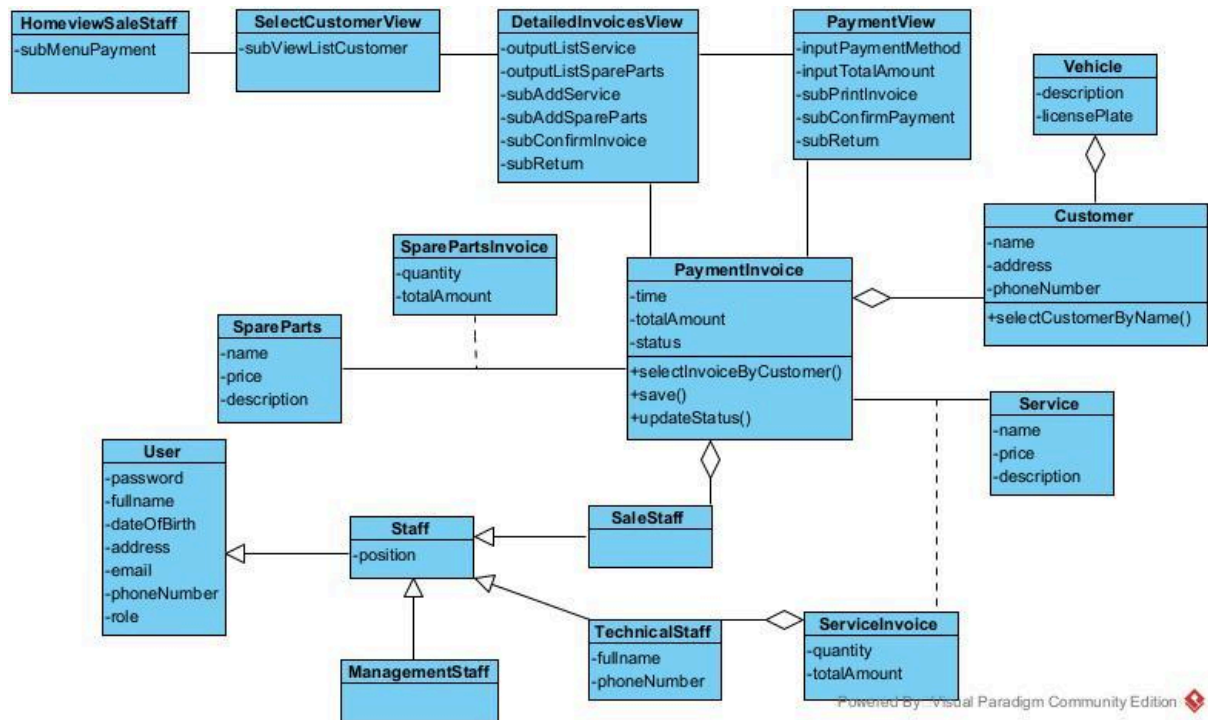
Classes diagram for Search for Service Information



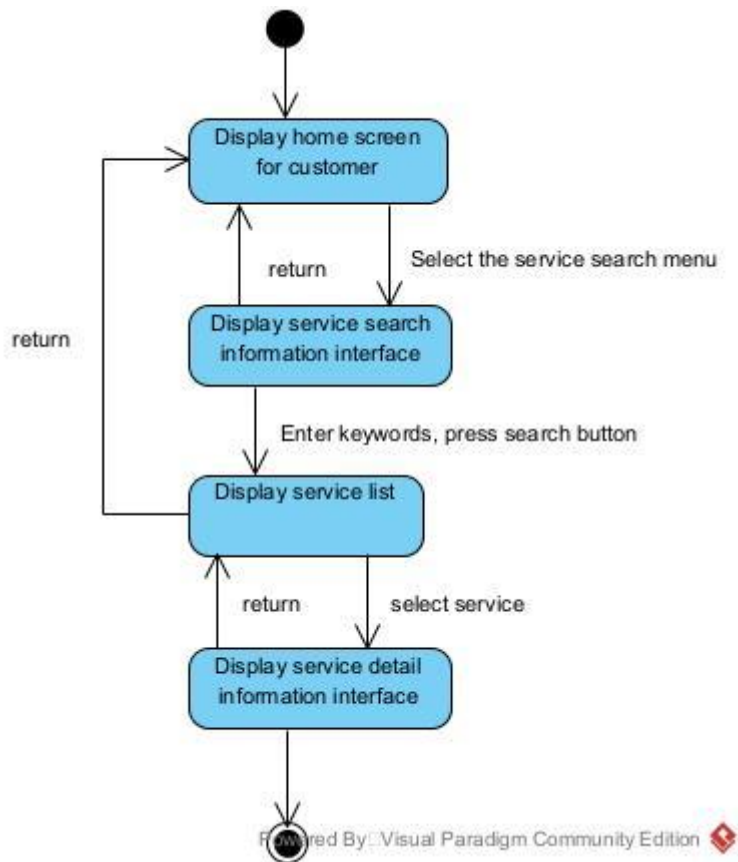
Classes diagram for Search for Spare parts Information



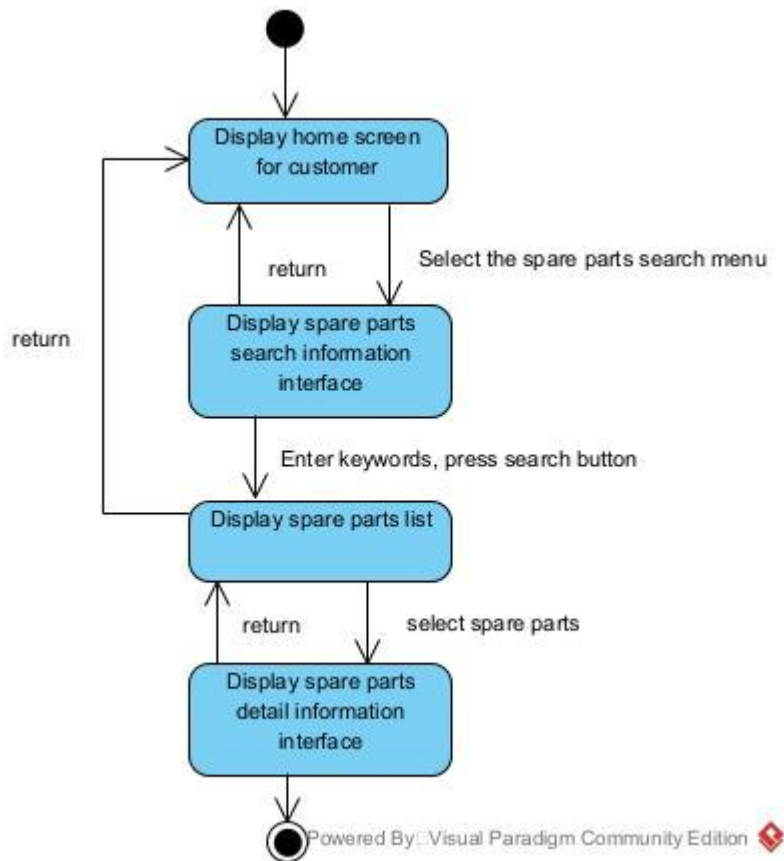
Classes diagram for Receive payment



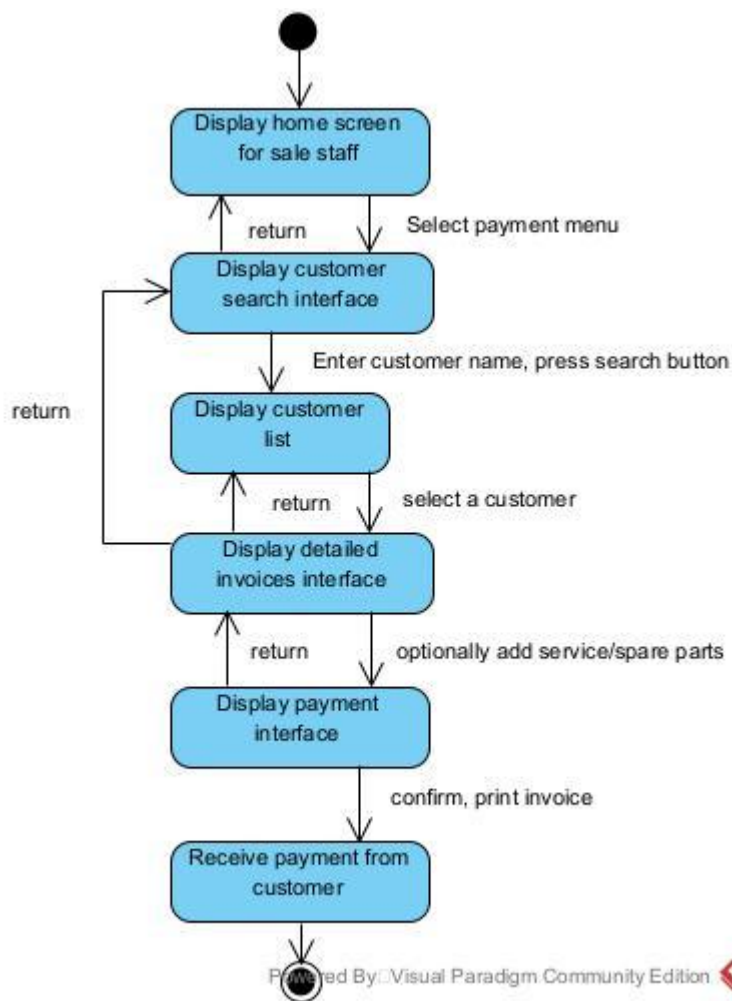
State Diagram for Search for Service Information



State Diagram for Search for Spare parts Information



State Diagram for Receive payment



4. Communication diagram and sequence diagram

Scenario 2 for searching for service information

• Search for Service Information

1. On the main interface of the customer after logging in, the customer selects the menu to search for service information.

2. The class **HomeviewCustomer** calls the class **SearchForServiceInformationView**.

3. The class **SearchForServiceInformationView** is displayed to the customer.

4. The customer enters a keyword into the search box and clicks the search button.

5. **SearchForServiceInformationView** calls the class **Service** to request a list of services whose names contain the entered keyword.

6. The class **Service** retrieves the list of services.

7. The class **Service** returns the result to **SearchForServiceInformationView**.

8. The class **SearchForServiceInformationView** displays the service list to the customer.

9. The customer selects a service from the list.

SearchForServiceInformationView calls the class **ServiceDetailsView**.

10. The class **ServiceDetailsView** calls the class **Service** to get the detailed information of the selected service by its ID.

11. The class **Service** retrieves the detailed information of the service.

12. The class **Service** returns the result to **ServiceDetailsView**.

13. The class **ServiceDetailsView** displays the information to the customer.

- **Search for Spare parts Information**

1. On the main interface of the customer after logging in, the customer selects the menu to search for spare part information.

2. The class **HomeviewCustomer** calls the class **SearchForSparePartsInformationView**.

3. The class **SearchForSparePartsInformationView** is displayed to the customer.

4. The customer enters a keyword into the search box and clicks the search button.

5. SearchForSparePartsInformationView calls the class **SpareParts** to request a list of spare parts whose names contain the entered keyword.

6. The class **SpareParts** retrieves the list of spare parts.

7. The class **SpareParts** returns the result to **SearchForSparePartsInformationView**.

8. The class **SearchForSparePartsInformationView** displays the spare parts list to the customer.

9. The customer selects a spare part from the list.

10. SearchForSparePartsInformationView calls the class **SparePartsDetailsView**.

11. The class **SparePartsDetailsView** calls the class **SpareParts** to get the detailed information of the selected spare part by its ID.

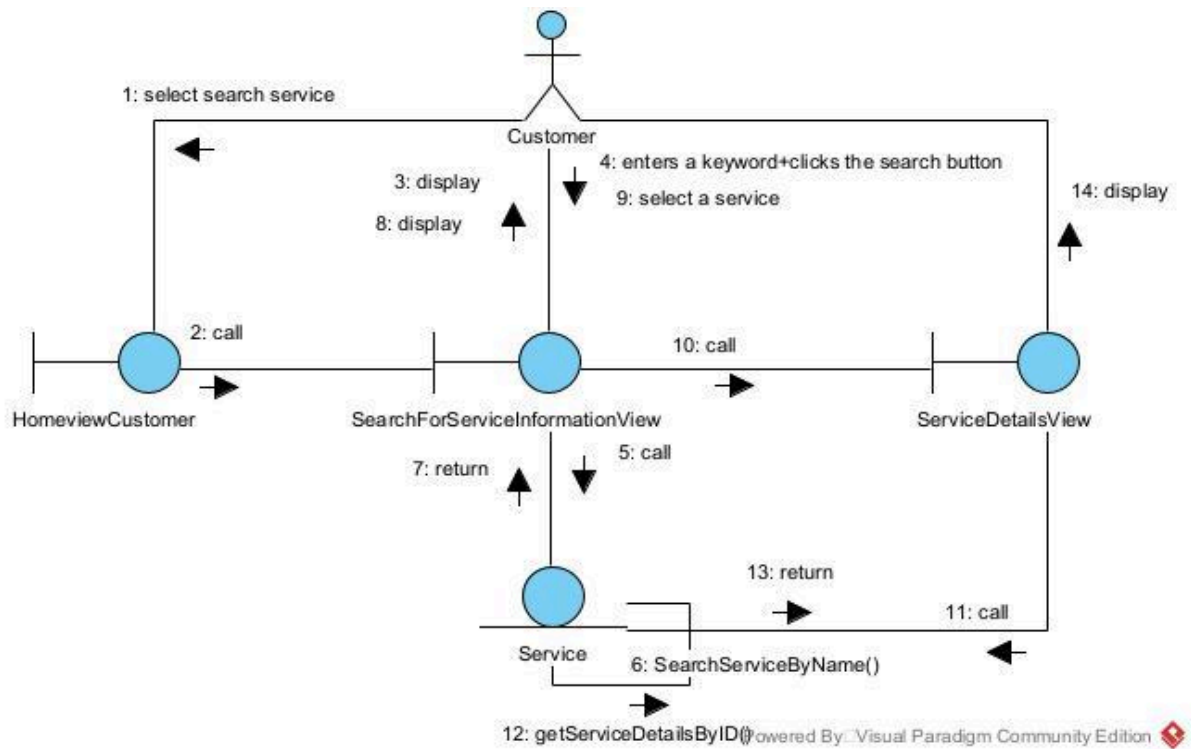
12. The class **SpareParts** retrieves the detailed information of the spare part.

13. The class **SpareParts** returns the result to **SparePartsDetailsView**.

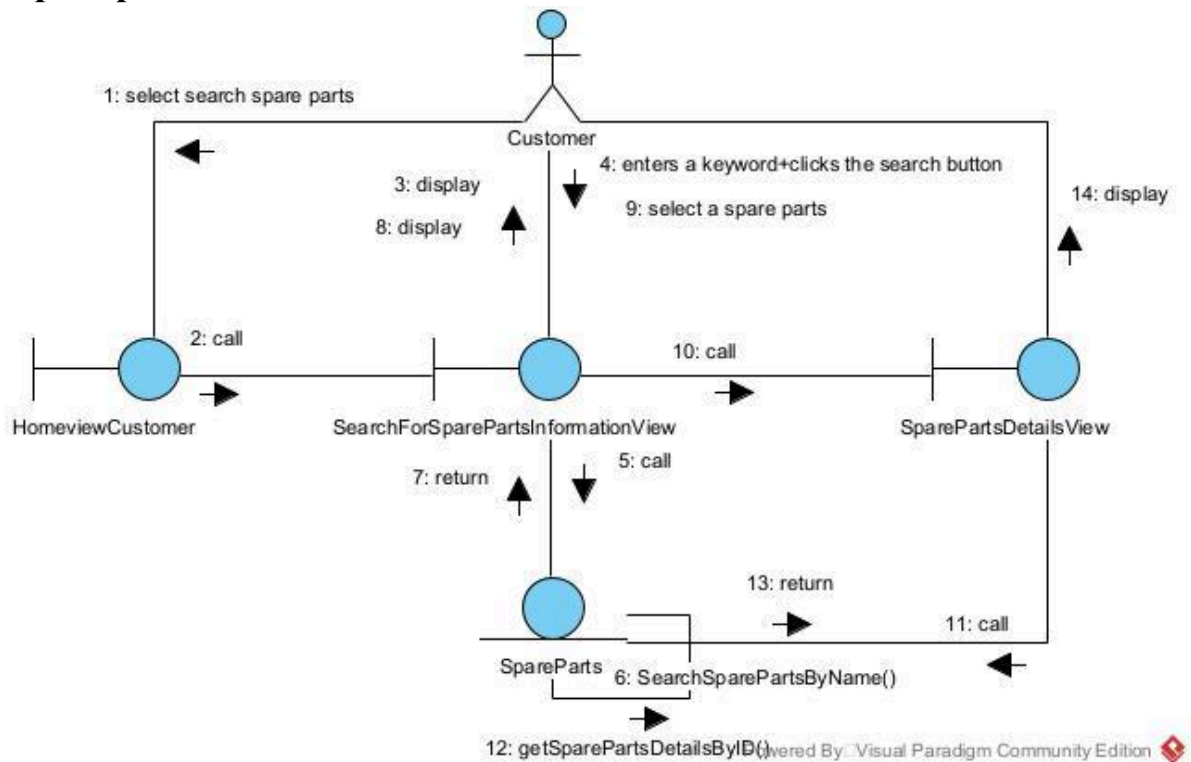
14. The class **SparePartsDetailsView** displays the information to the customer.

Communication Diagram for Search Service Information

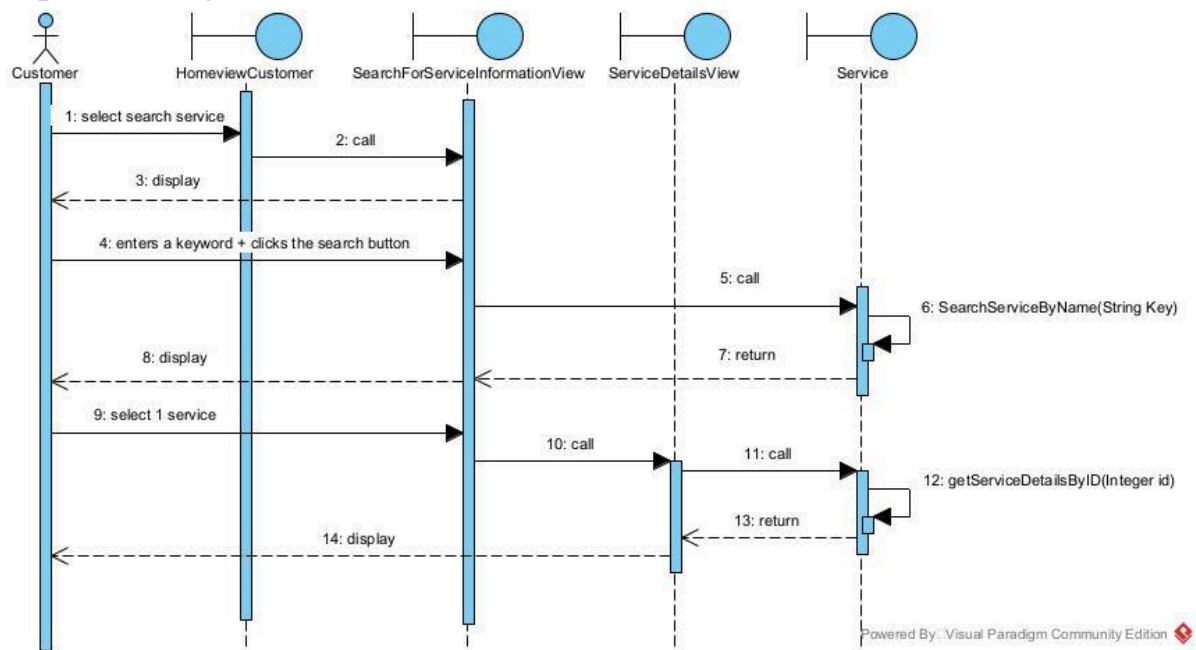
* Service



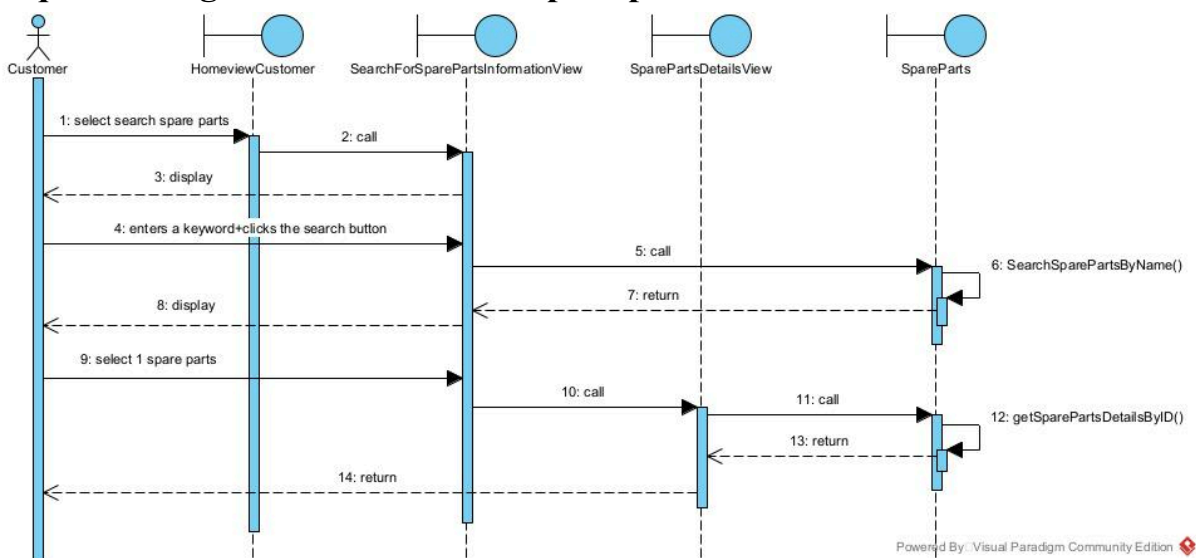
* Spare parts



Sequence diagram for Search for Service Information



Sequence diagram for Search for Spare parts Information

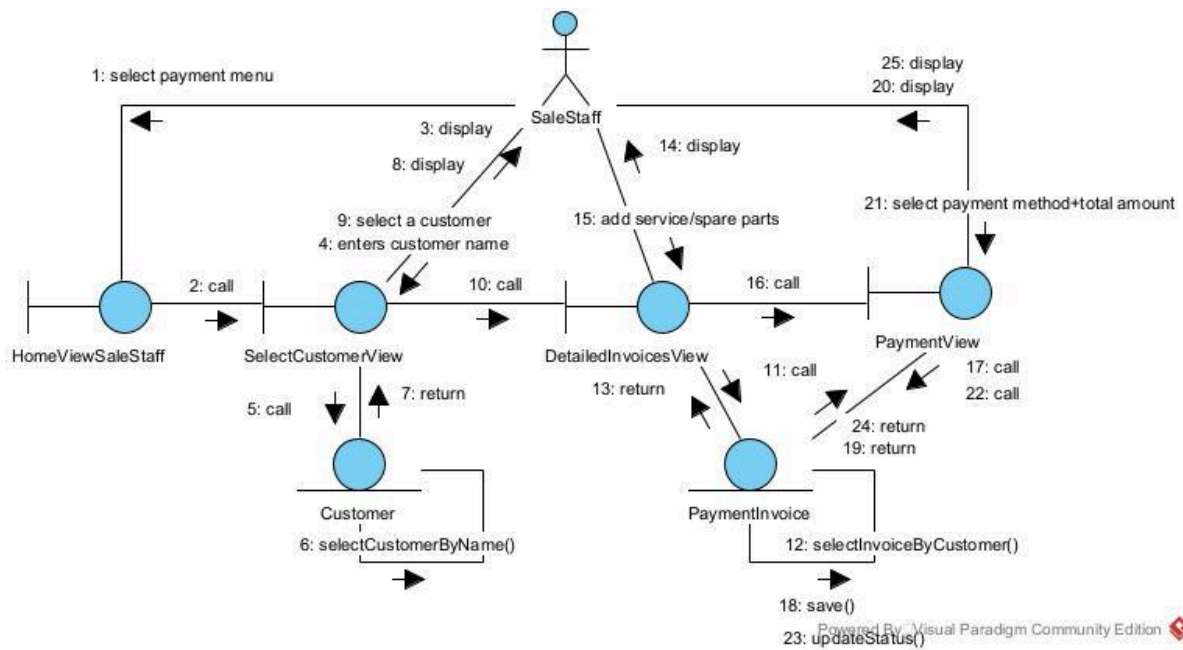


Scenario 2 for Receive payment

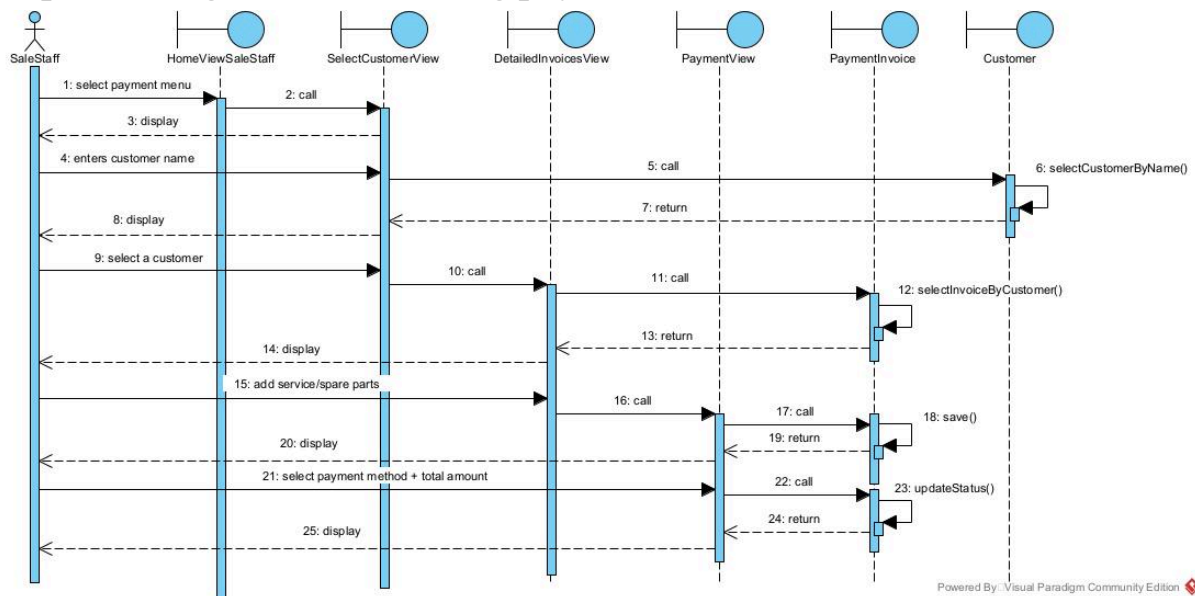
1. The SaleStaff selects the payment menu option on the interface.
2. The **HomeViewSaleStaff** system calls the **SelectCustomerView** screen.
3. The **SelectCustomerView** displays the customer selection interface to the Staff.
4. The Staff enters the customer name for searching.
5. **SelectCustomerView** calls the **selectCustomerByName()** function in the Customer object to perform the search.
6. The Customer object executes the search.

7. The **Customer** object returns the list of found customers to **SelectCustomerView**.
8. **SelectCustomerView** displays the list of customers on the interface.
9. The Staff selects a specific Customer from the list.
10. **SelectCustomerView** calls the **DetailedInvoicesView** screen.
11. **DetailedInvoicesView** calls the **selectInvoiceByCustomer()** function in the **PaymentInvoice** object to retrieve the customer's pending invoice information.
12. The **PaymentInvoice** object executes the invoice search.
13. **PaymentInvoice** returns the detailed invoice information to **DetailedInvoicesView**.
14. **DetailedInvoicesView** displays the detailed invoice information to the Staff.
15. The Staff adds services/spare parts to the invoice.
16. **DetailedInvoicesView** calls the **save()** function in the **PaymentInvoice** object to persist the changes.
17. The **PaymentInvoice** object executes the saving process.
18. **PaymentInvoice** returns the successful save result.
19. **DetailedInvoicesView** displays the save confirmation message.
20. The Staff selects the payment method and confirms the total amount.
21. **DetailedInvoicesView** calls the **PaymentView** screen to handle the payment.
22. **PaymentView** calls the **updateStatus()** function in the **PaymentInvoice** object to update the invoice status to Paid.
23. The **PaymentInvoice** object executes the status update.
24. **PaymentInvoice** returns the status update result.
25. **PaymentView** displays the successful payment confirmation.

Communication Diagram for Receive Payment



Sequence diagram for Receiving payment

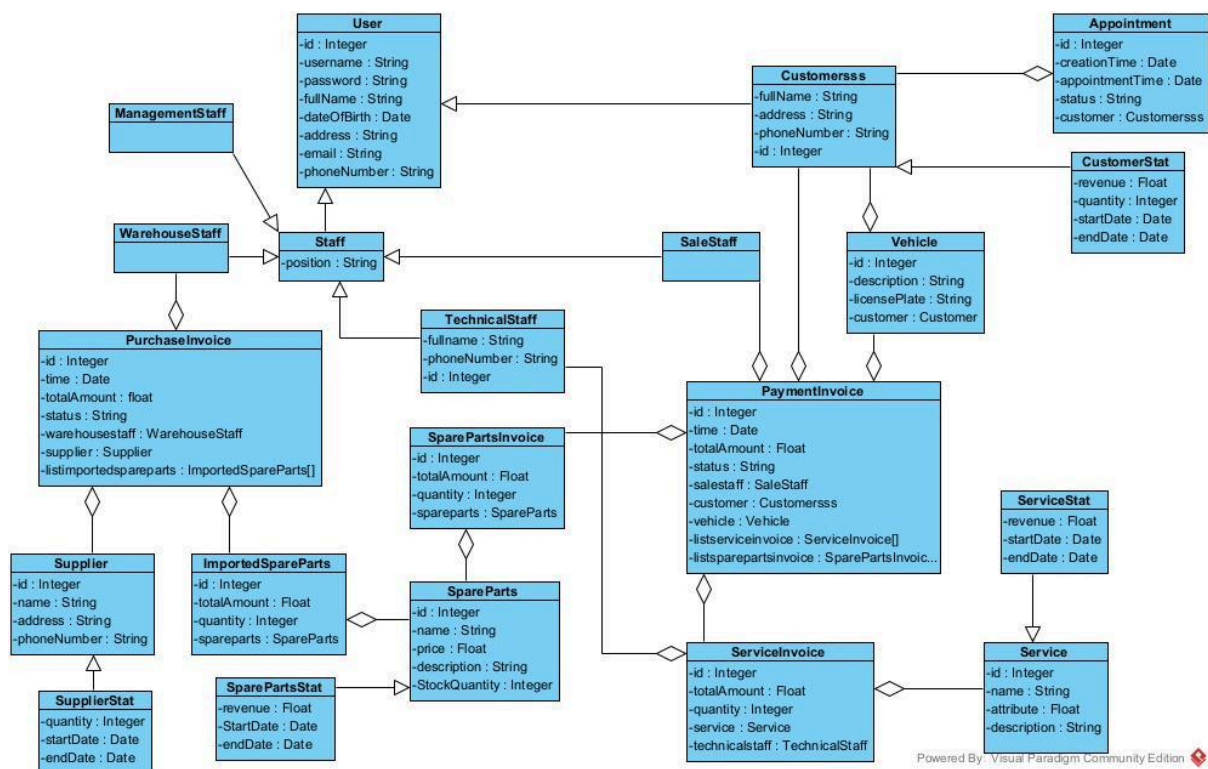


C. Design

1. Entity class diagram

- Classes are supplemented with the id attribute: except classes Stat, Customer, SaleStaff, TechnicalStaff, WarehouseStaff, ManagementStaff.

- Add data types for the attributes.
- The relationship PaymentInvoice– SpareParts → SparePartsInvoice is changed so that SparePartsInvoice contains SpareParts.
- The relationship PaymentInvoice – Service → ServiceInvoice is changed so that ServiceInvoice contains Service.
- PaymentInvoice contains a list of ServiceInvoice and a list of SparePartsInvoice.
- The relationship PurchaseInvoice – SpareParts → ImportedSpareParts is changed so that ImportedSpareParts contains SpareParts.
- PurchaseInvoice contains a list of ImportedSpareParts.



2. Database Design

Class **User** → **User**

Class **Staff** → **tblStaff**

Class **Vehicle** → **tblVehicle**

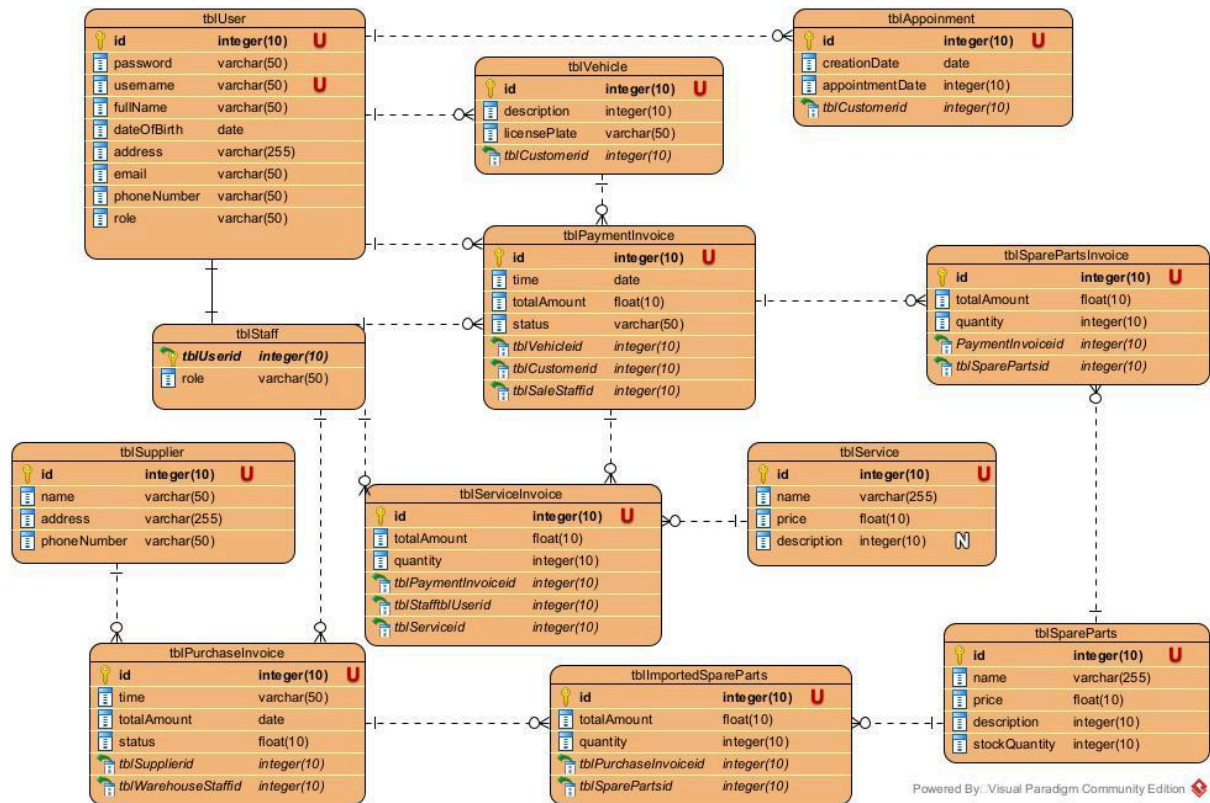
Class **Appointment** → **tblAppointment**

Class **PaymentInvoice** → **tblPaymentInvoice**

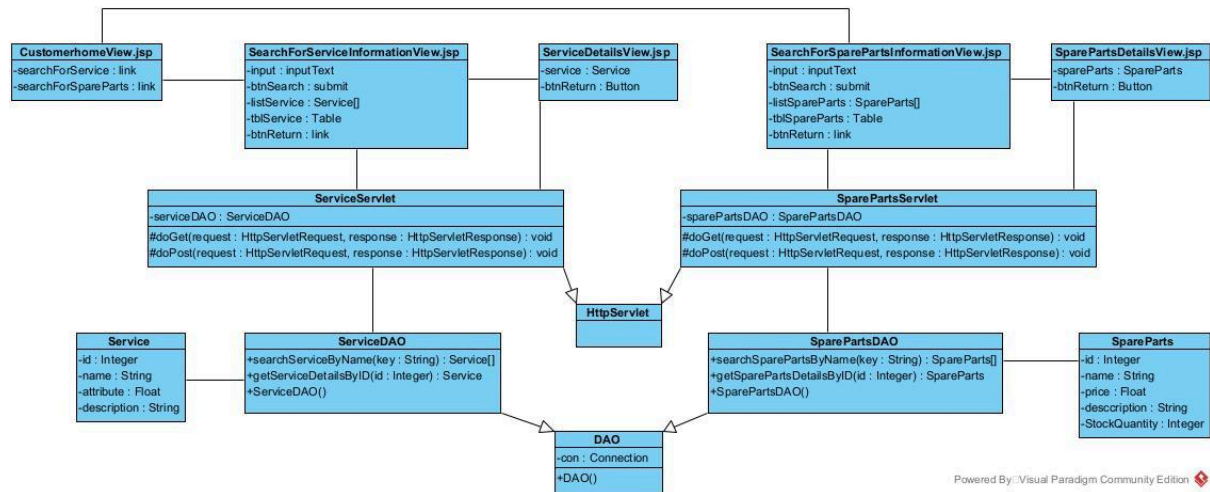
Class **ServiceInvoice** → **tblServiceInvoice**
Class **SparePartsInvoice** → **tblSparePartsInvoice**
Class **Service** → **tblService**
Class **SpareParts** → **tblSpareParts**
Class **Supplier** → **tblSupplier**
Class **PurchaseInvoice** → **tblPurchaseInvoice**
Class **ImportedSpareParts** → **tblImportedSpareParts**

Convert cardinality relationships between entity classes into cardinality relationships between tables:

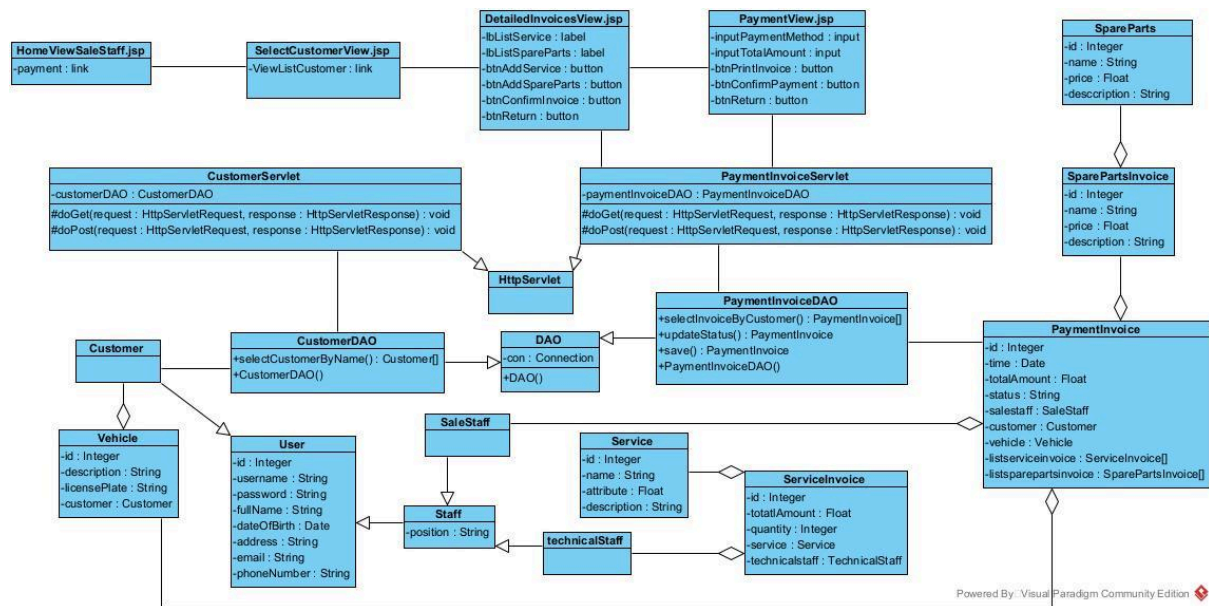
- **1 tblUser – n tblAppointment → tblAppointment** contains foreign key *CustomerId*.
- **1 tblStaff – n tblVehicle → tblVehicle** contains foreign key *CustomerId*.
- **1 tblUser – 1 tblStaff → tblStaff** contains foreign key *tblUserId*.
- **1 tblUser – n tblPaymentInvoice → tblPaymentInvoice** contains foreign key *CustomerId*.
- **1 tblStaff – n tblPaymentInvoice → tblPaymentInvoice** contains foreign key *tblSaleStaffId*.
- **1 tblVehicle – n tblPaymentInvoice → tblPaymentInvoice** contains foreign key *tblVehicleId*.
- **1 tblPaymentInvoice – n tblSparePartsInvoice → tblSparePartsInvoice** contains foreign key *tblPaymentInvoiceId*.
- **1 tblPaymentInvoice – n tblServiceInvoice → tblServiceInvoice** contains foreign key *tblPaymentInvoiceId*.
- **1 tblSpareParts – n tblSparePartsInvoice → tblSparePartsInvoice** contains foreign key *tblSparePartsId*.
- **1 tblService – n tblServiceInvoice → tblServiceInvoice** contains foreign key *tblServiceId*.
- **1 tblPurchaseInvoice – n tblImportedSpareParts → tblImportedSpareParts** contains foreign key *tblPurchaseInvoiceId*.
- **1 tblStaff – n tblPurchaseInvoice → tblPurchaseInvoice** contains foreign key *tblWarehouseStaffId*.
- **1 tblSupplier – n tblPurchaseInvoice → tblPurchaseInvoice** contains foreign key *tblSupplierId*.
- **1 tblSpareParts – n tblImportedSpareParts → tblImportedSpareParts** contains foreign key *tblSparePartsId*.



3. Class diagram for Search for Service information

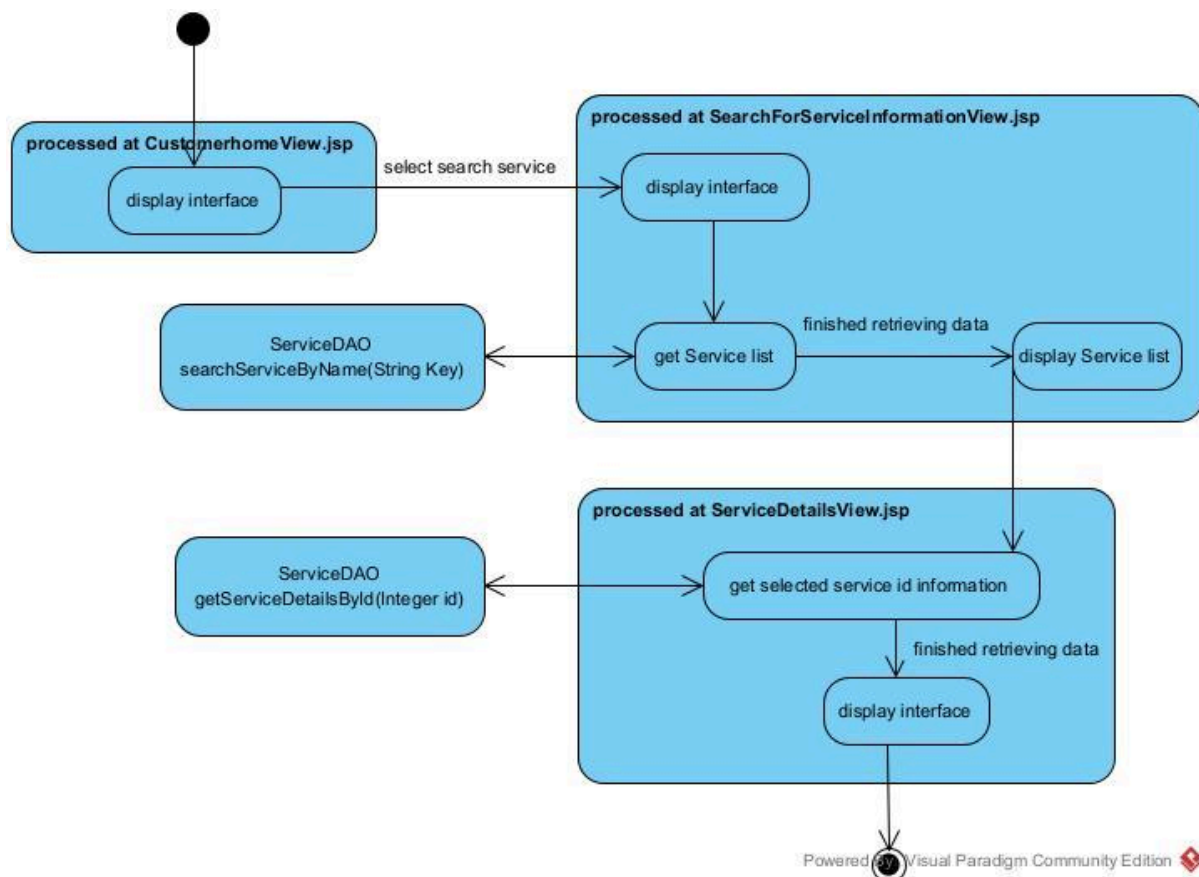


4. Class diagram for Receive Payment

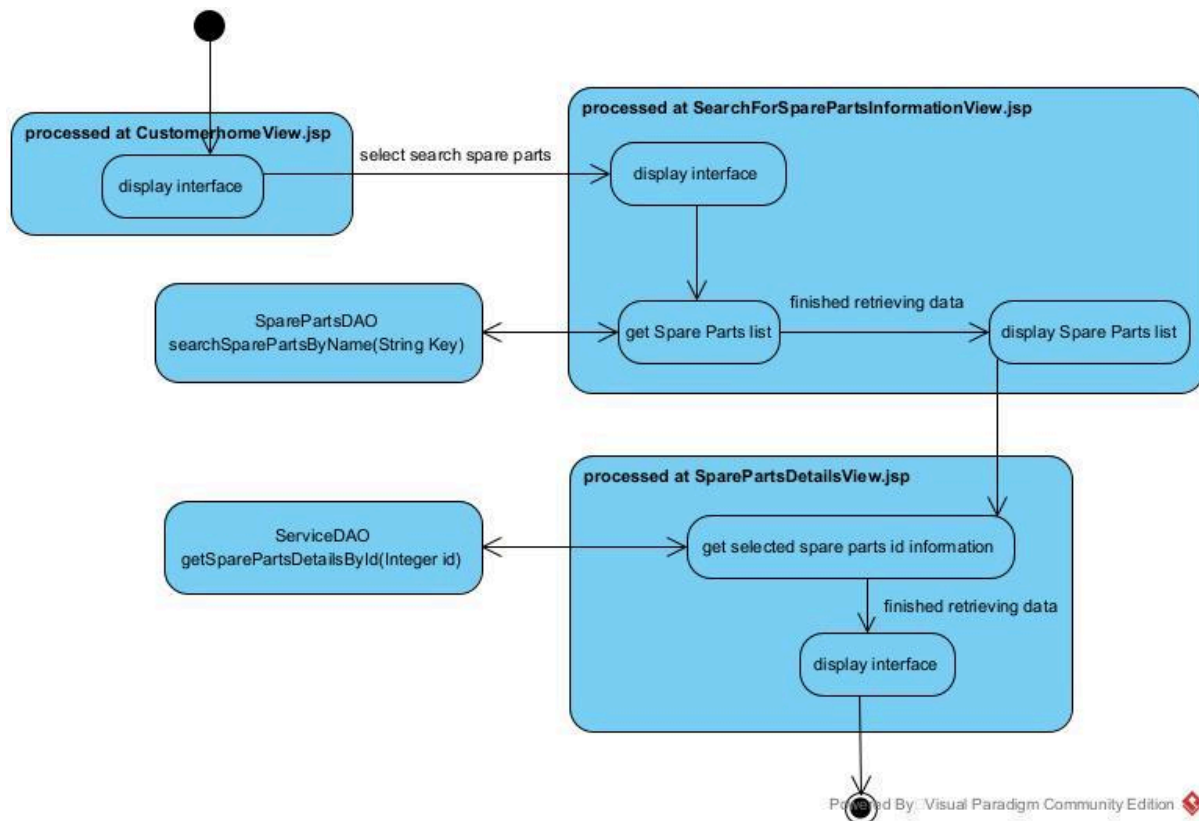


5. Activity for Search for Service information

5.1. Service



5.2. SpareParts



6. Scenario v3 for Search for Service information

- **Service**

1. On the page **HomeViewCustomer.jsp**, after successfully logging in, the customer selects the “Search Service” function.
2. The page **HomeViewCustomer.jsp** calls the page **SearchForServiceInformationView.jsp**.
3. The page **SearchForServiceInformationView.jsp** is displayed to the customer.
4. The customer enters a keyword into the search box and clicks the “Search” button.
5. The page **SearchForServiceInformationView.jsp** sends a **GET** request to the class **ServiceServlet** to search for services.
6. The class **ServiceServlet** executes the **doGet(req, res)** method.

7. The class **ServiceServlet** calls the class **ServiceDAO** using the function **searchServiceByName()** to find services whose names contain the entered keyword.
8. The function **searchServiceByName()** executes and calls the class **Service** to encapsulate the information.
9. The class **Service** packages the entity information.
10. The class **Service** returns the result to the function **searchServiceByName()** in the class **ServiceDAO**.
11. The class **ServiceDAO** returns the result to the class **ServiceServlet**.
12. The class **ServiceServlet** returns the list of matched services to the page **SearchForServiceInformationView.jsp**.
13. The page **SearchForServiceInformationView.jsp** displays the results to the customer.
14. The customer clicks on a service.
15. The page **SearchForServiceInformationView.jsp** calls the page **ServiceDetailsView.jsp**.
16. The page **ServiceDetailsView.jsp** sends a **GET request** to the class **ServiceServlet** to request details of the selected service.
17. The class **ServiceServlet** executes the **doGet(req, res)** method.
18. The class **ServiceServlet** calls the class **ServiceDAO** using the function **getServiceDetailsByID()** (by ID).
19. The function **getServiceDetailsByID()** executes and calls the class **Service** to encapsulate the information.
20. The class **Service** packages the entity information.
21. The class **Service** returns the result to the function **getServiceDetailsByID()** in the class **ServiceDAO**.
22. The class **ServiceDAO** returns the result to the class **ServiceServlet**.
23. The class **ServiceServlet** returns the service details to the page **ServiceDetailsView.jsp**.

24. The page **ServiceDetailsView.jsp** displays the service information to the customer.

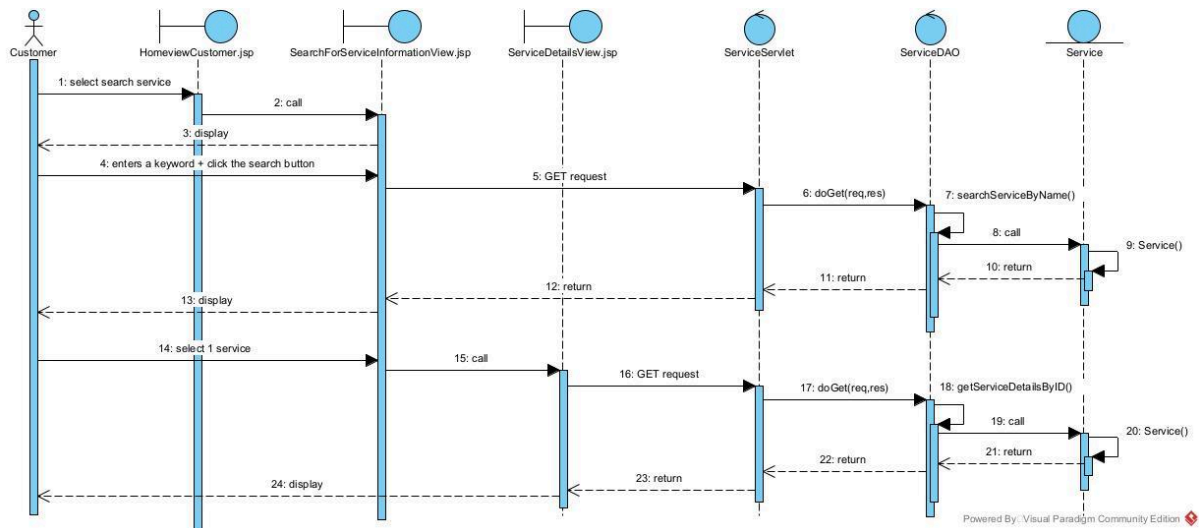
- **Spare Parts**

1. On the page **HomeViewCustomer.jsp**, after successfully logging in, the customer selects the “Search SpareParts” function.
2. The page **HomeViewCustomer.jsp** calls the page **SearchForSparePartsInformationView.jsp**.
3. The page **SearchForSparePartsInformationView.jsp** is displayed to the customer.
4. The customer enters a keyword into the search box and clicks the “Search” button.
5. The page **SearchForSparePartsInformationView.jsp** sends a **GET request** to the class **SparePartsServlet** to search for spare parts.
6. The class **SparePartsServlet** executes the **doGet(req, res)** method.
7. The class **SparePartsServlet** calls the class **SparePartsDAO** using the function **searchSparePartsByName()** to find spare parts whose names contain the entered keyword.
8. The function **searchSparePartsByName()** executes and calls the class **SpareParts** to encapsulate the information.
9. The class **SpareParts** packages the entity information.
10. The class **SpareParts** returns the result to the function **searchSparePartsByName()** in the class **SparePartsDAO**.
11. The class **SparePartsDAO** returns the result to the class **SparePartsServlet**.
12. The class **SparePartsServlet** returns the list of matched spare parts to the page **SearchForSparePartsInformationView.jsp**.
13. The page **SearchForSparePartsInformationView.jsp** displays the results to the customer.
14. The customer clicks on a spare part.

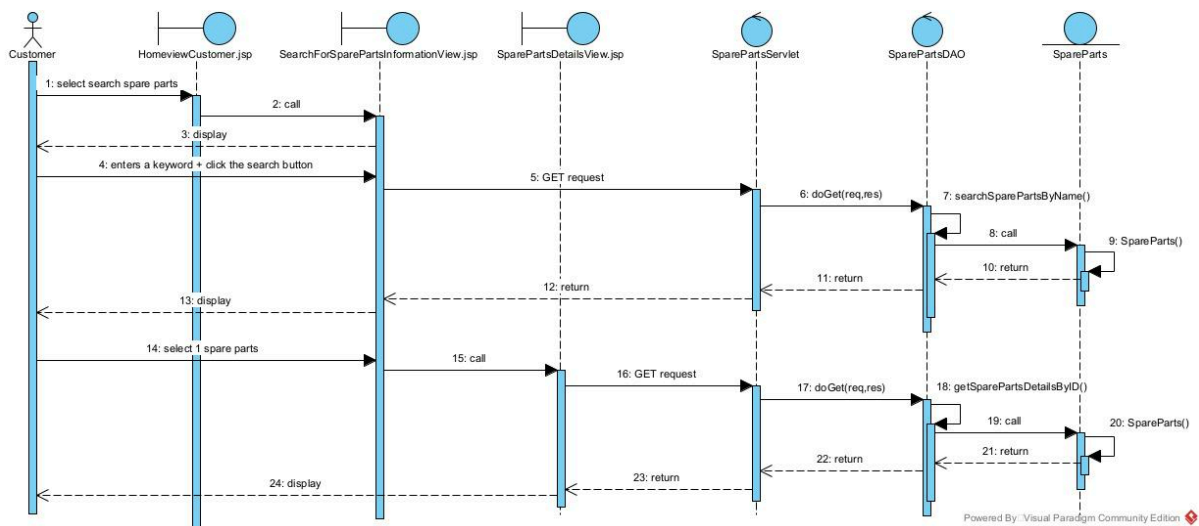
15. The page **SearchForSparePartsInformationView.jsp** calls the page **SparePartsDetailsView.jsp**.
16. The page **SparePartsDetailsView.jsp** sends a **GET request** to the class **SparePartsServlet** to request details of the selected spare part.
17. The class **SparePartsServlet** executes the **doGet(req, res)** method.
18. The class **SparePartsServlet** calls the class **SparePartsDAO** using the function **getSparePartsDetailsByID()** (by ID).
19. The function **getSparePartsDetailsByID()** executes and calls the class **SpareParts** to encapsulate the information.
20. The class **SpareParts** packages the entity information.
21. The class **SpareParts** returns the result to the function **getSparePartsDetailsByID()** in the class **SparePartsDAO**.
22. The class **SparePartsDAO** returns the result to the class **SparePartsServlet**.
23. The class **SparePartsServlet** returns the spare part details to the page **SparePartsDetailsView.jsp**.
24. The page **SparePartsDetailsView.jsp** displays the spare part information to the customer.

7. Sequence for Search for Service information

- Service



• SpareParts



8. Scenario v3 for Receive Payment

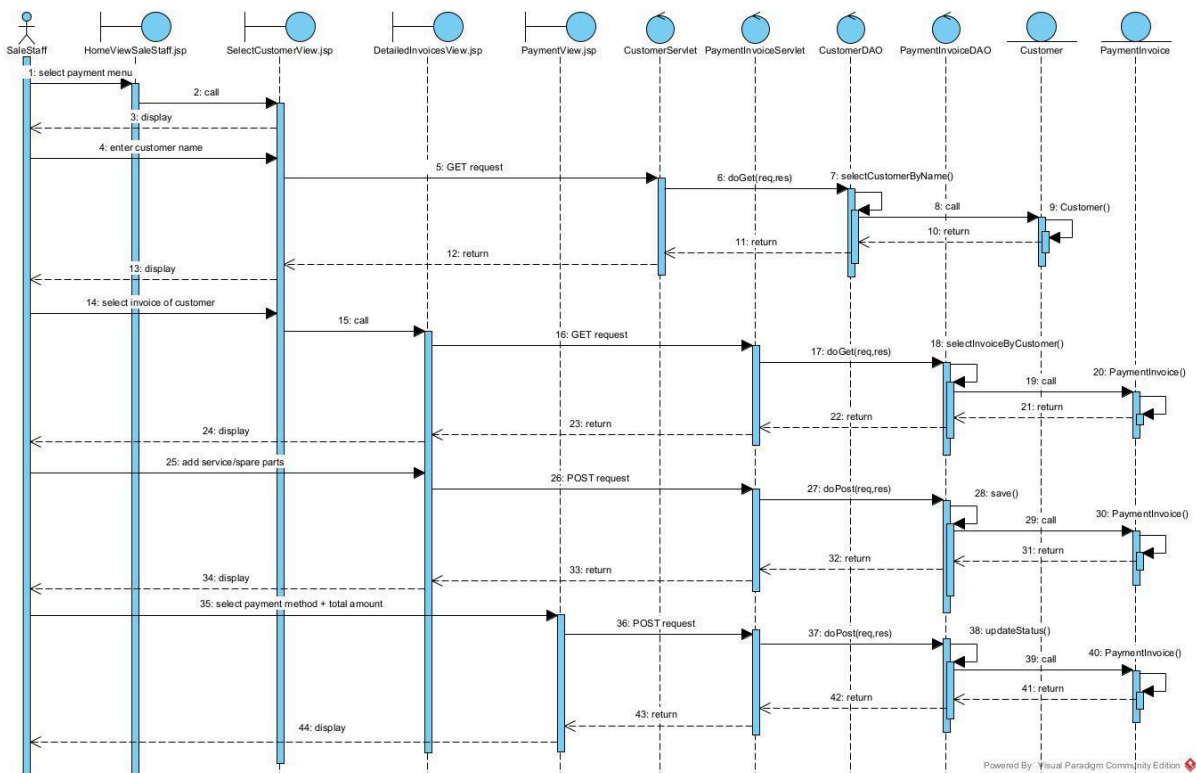
1. At the page **HomeViewSaleStaff.jsp**, after logging in successfully, the sale staff selects the "Payment" function.
2. The page **HomeViewSaleStaff.jsp** calls the page **SelectCustomerView.jsp**.
3. The page **SelectCustomerView.jsp** is displayed to the sale staff.
4. The sale staff enters the customer's name into the search box and clicks the "Search" button.

5. The page **SelectCustomerView.jsp** sends a **GET request** to the class **CustomerServlet** to find the customers whose names contain the entered keyword.
6. The class **CustomerServlet** executes the **doGet()** method.
7. The class **CustomerServlet** calls the class **CustomerDAO** using the function **selectCustomerByName()**.
8. The function **selectCustomerByName()** executes and calls the class **Customer** to encapsulate the customer information.
9. The class **Customer** packages the customer entity information.
10. The class **Customer** returns the result to the function **selectCustomerByName()** in the class **CustomerDAO**.
11. The class **CustomerDAO** returns the list of matched customers to the class **CustomerServlet**.
12. The class **CustomerServlet** returns the list of matched customers to the page **SelectCustomerView.jsp**.
13. The page **SelectCustomerView.jsp** displays the list of matched customers to the sales staff.
14. The sales staff clicks to select a customer's invoice.
15. The page **SelectCustomerView.jsp** calls the page **DetailedInvoicesView.jsp**.
16. The page **DetailedInvoicesView.jsp** sends a **GET request** to the class **PaymentInvoiceServlet** to request the invoice details of the selected customer.
17. The class **PaymentInvoiceServlet** executes the **doGet()** method.
18. The class **PaymentInvoiceServlet** calls the class **PaymentInvoiceDAO** using the function **selectInvoiceByCustomer()**.
19. The function **selectInvoiceByCustomer()** executes and calls the class **PaymentInvoice** to encapsulate the invoice information.
20. The class **PaymentInvoice** packages the invoice entity information, including a list of services and spare parts used.
21. The class **PaymentInvoice** returns the result to the function **selectInvoiceByCustomer()** in the class **PaymentInvoiceDAO**.

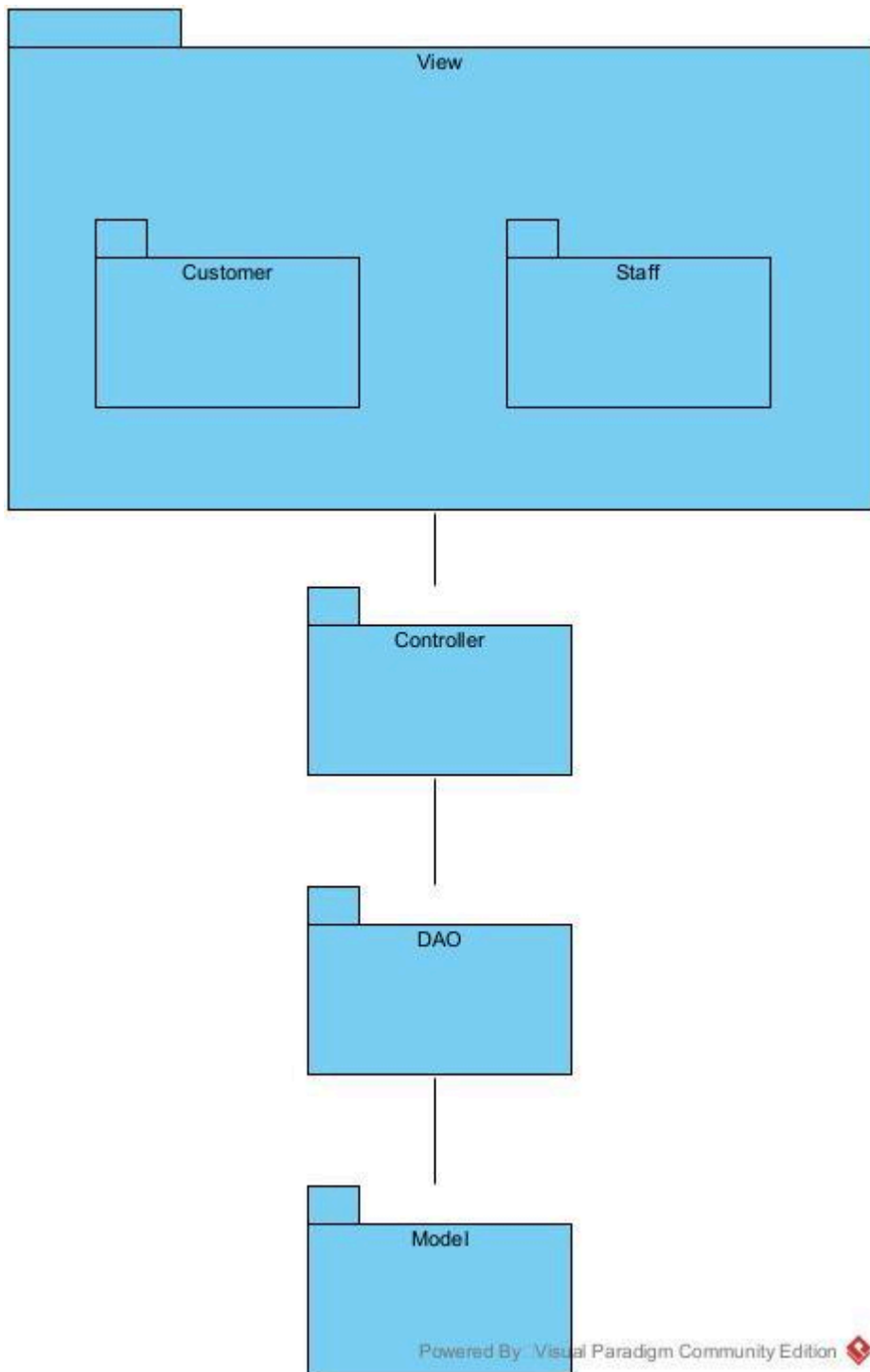
22. The class **PaymentInvoiceDAO** returns the result to the class **PaymentInvoiceServlet**.
23. The class **PaymentInvoiceServlet** returns the result to the page **DetailedInvoicesView.jsp**.
24. The page **DetailedInvoicesView.jsp** displays the invoice details to the sale staff.
25. (Optional) The sale staff adds a service or a spare part.
26. The page **DetailedInvoicesView.jsp** sends a **POST request** to the class **PaymentInvoiceServlet** to record the added information.
27. The class **PaymentInvoiceServlet** executes the **doPost()** method.
28. The class **PaymentInvoiceServlet** calls the class **PaymentInvoiceDAO** using the function **save()**.
29. The function **save()** calls the class **PaymentInvoice** to save added data.
30. The class **PaymentInvoice** packages the added entity information.
31. The class **PaymentInvoice** returns the confirmation result to the function **save()** in the class **PaymentInvoiceDAO**.
32. The class **PaymentInvoiceDAO** returns the result to the class **PaymentInvoiceServlet**.
33. The class **PaymentInvoiceServlet** returns the result to the page **DetailedInvoicesView.jsp**.
34. The page **DetailedInvoicesView.jsp** displays the updated invoice to the sale staff.
35. The sales staff selects payment method and total amount.
36. The page **PaymentView.jsp** sends a **POST request** to the class **PaymentInvoiceServlet** to record the payment information.
37. The class **PaymentInvoiceServlet** executes the **doPost()** method.
38. The class **PaymentInvoiceServlet** calls the class **PaymentInvoiceDAO** using the function **updateStatus()**.
39. The function **updateStatus()** executes, updates the invoice status to “Paid,” and calls the class **PaymentInvoice** to encapsulate the payment data.

40. The class **PaymentInvoice** packages the payment entity information.
41. The class **PaymentInvoice** returns the confirmation result to the function **updateStatus()** in the class **PaymentInvoiceDAO**.
42. The class **PaymentInvoiceDAO** returns the result to the class **PaymentInvoiceServlet**.
43. The class **PaymentInvoiceServlet** returns the result to the page **PaymentView.jsp**.
44. The page **PaymentView.jsp** displays the payment result to the sales staff.

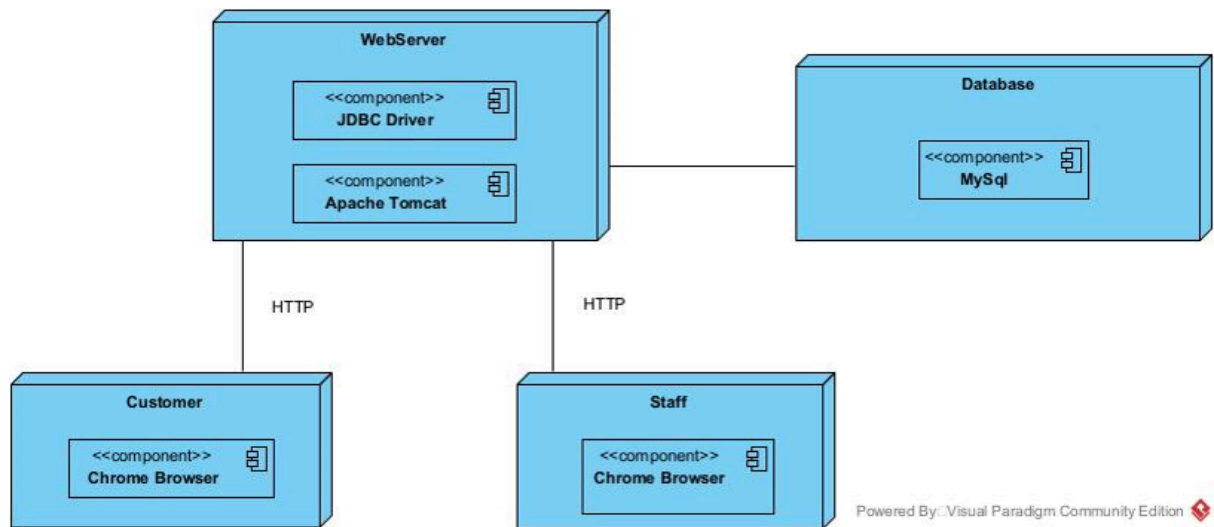
9. Sequence for Search for Receive Payment



10. Package diagram

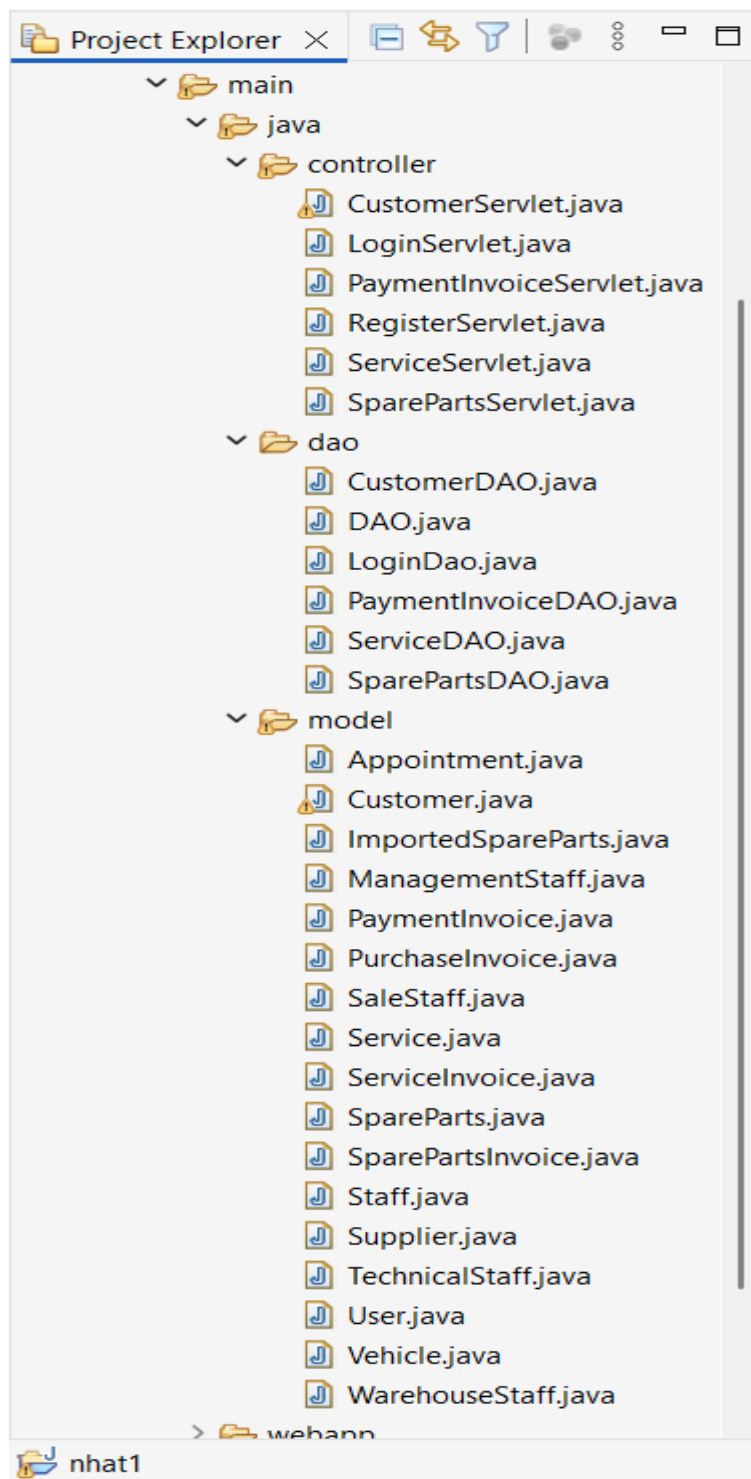


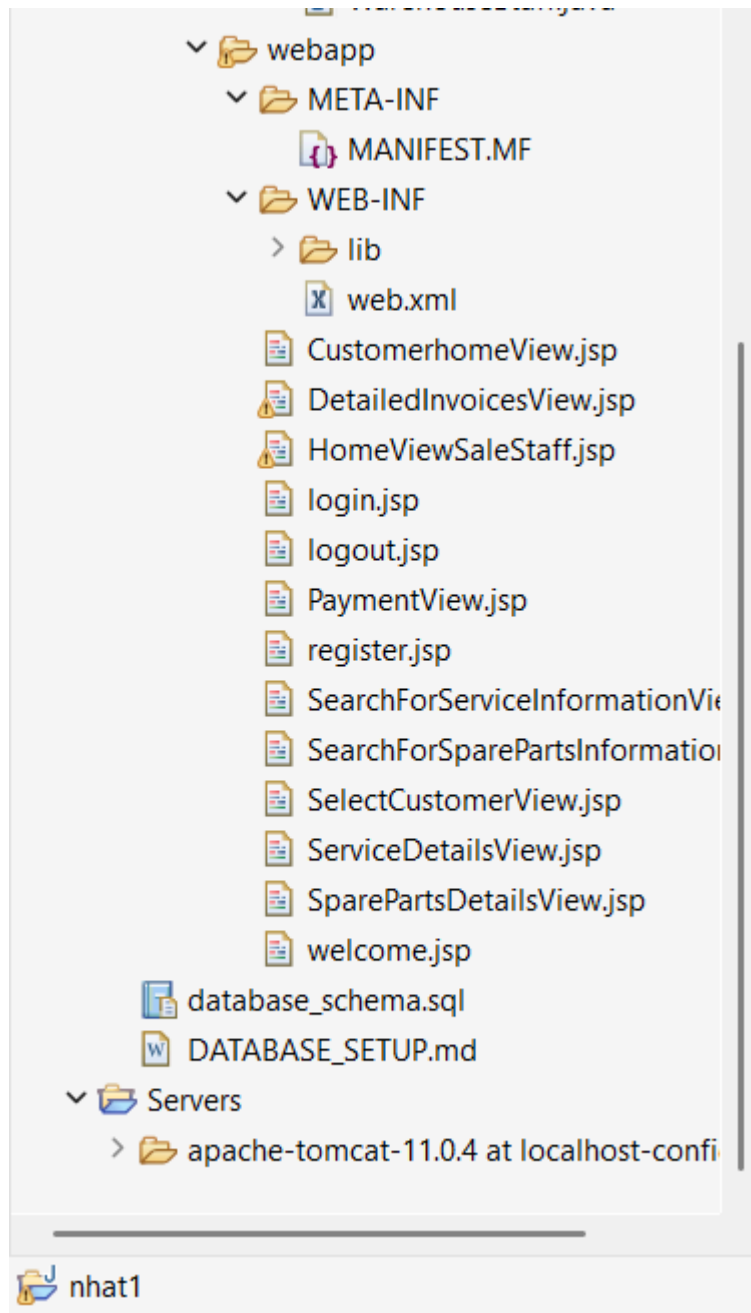
11. Deployment diagram



D. CODE

1. File Structure

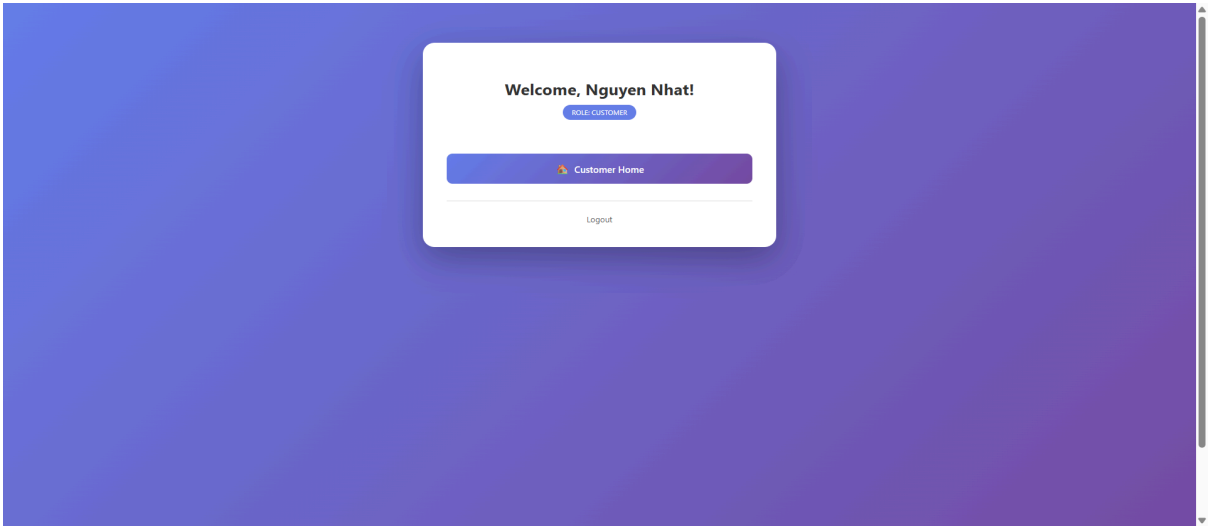




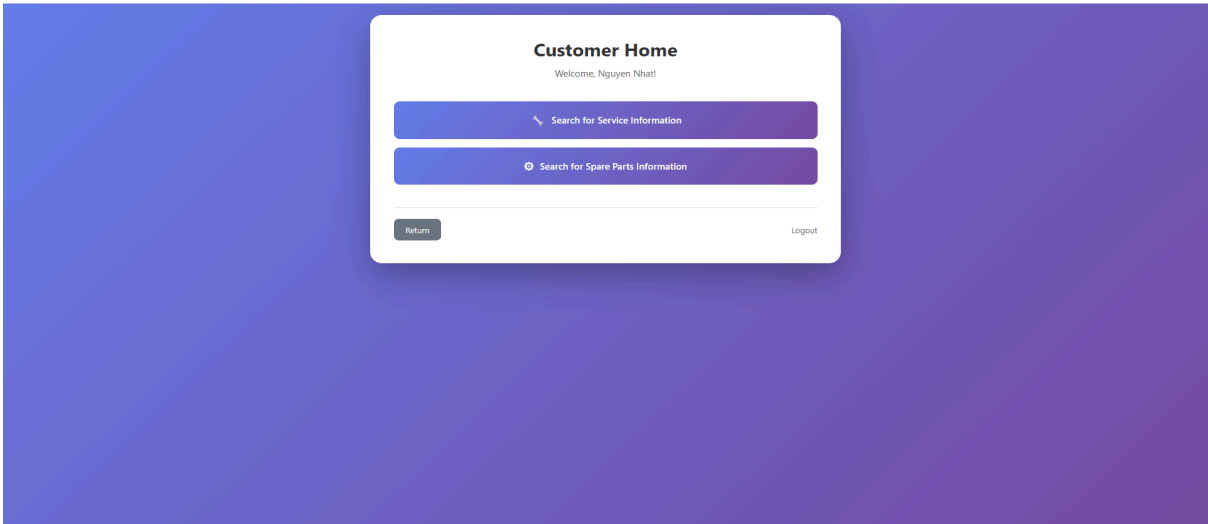
2. Interface

2.1. Module 1:

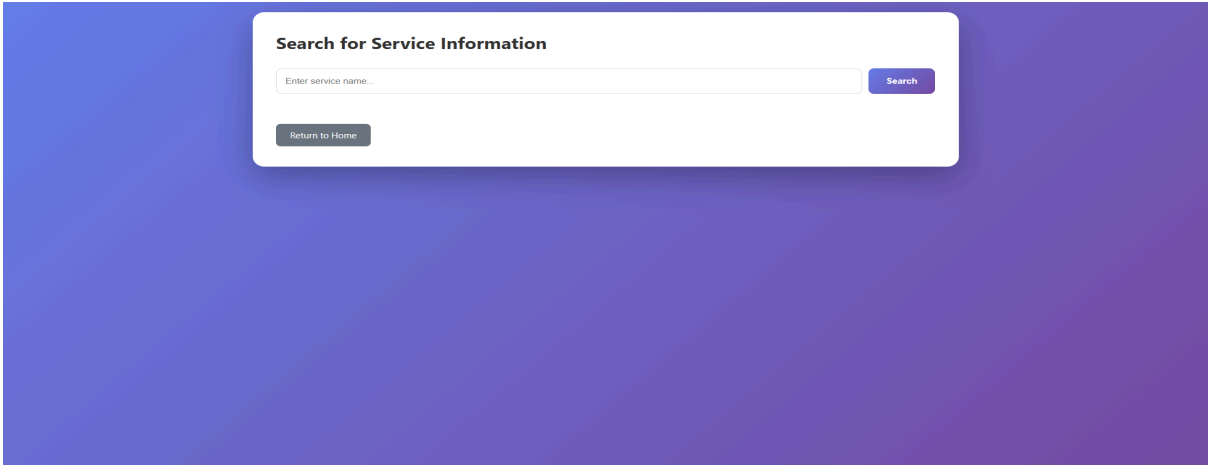
Welcome.jsp

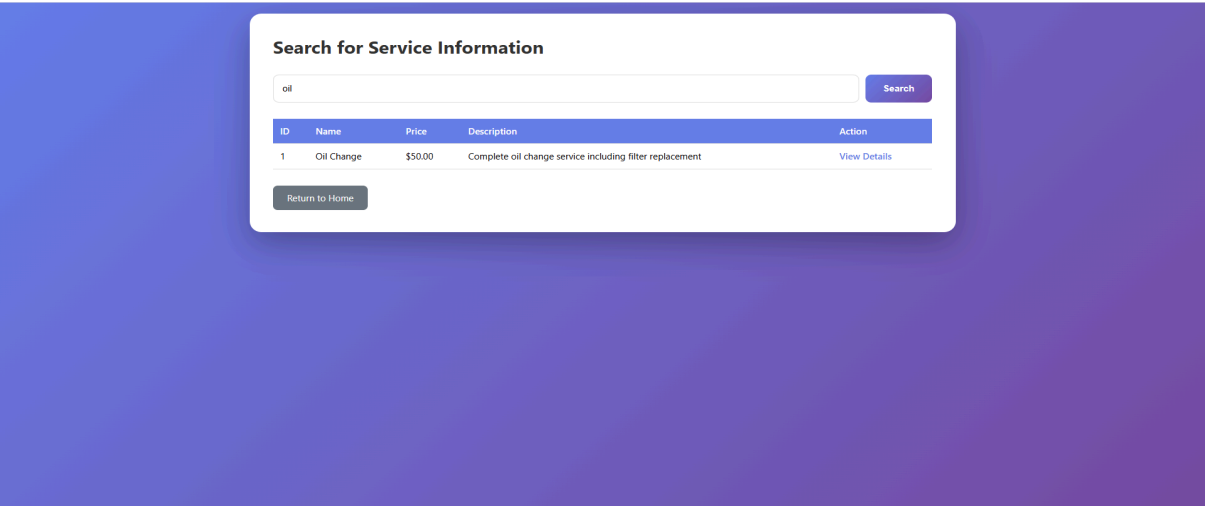


CustomerHomeView.jsp

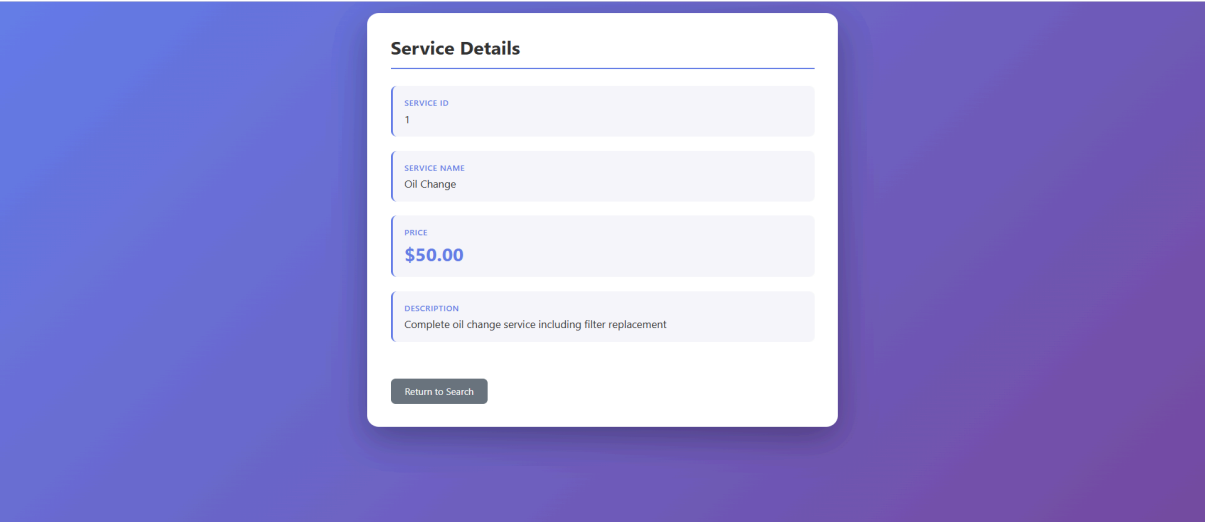


SearchForServiceInformationView.jsp

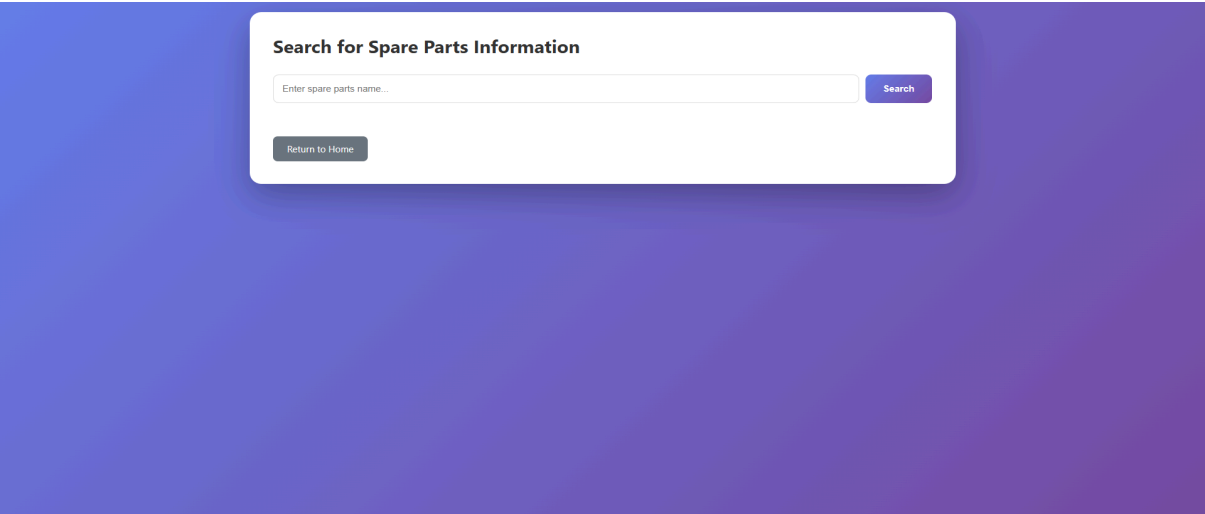


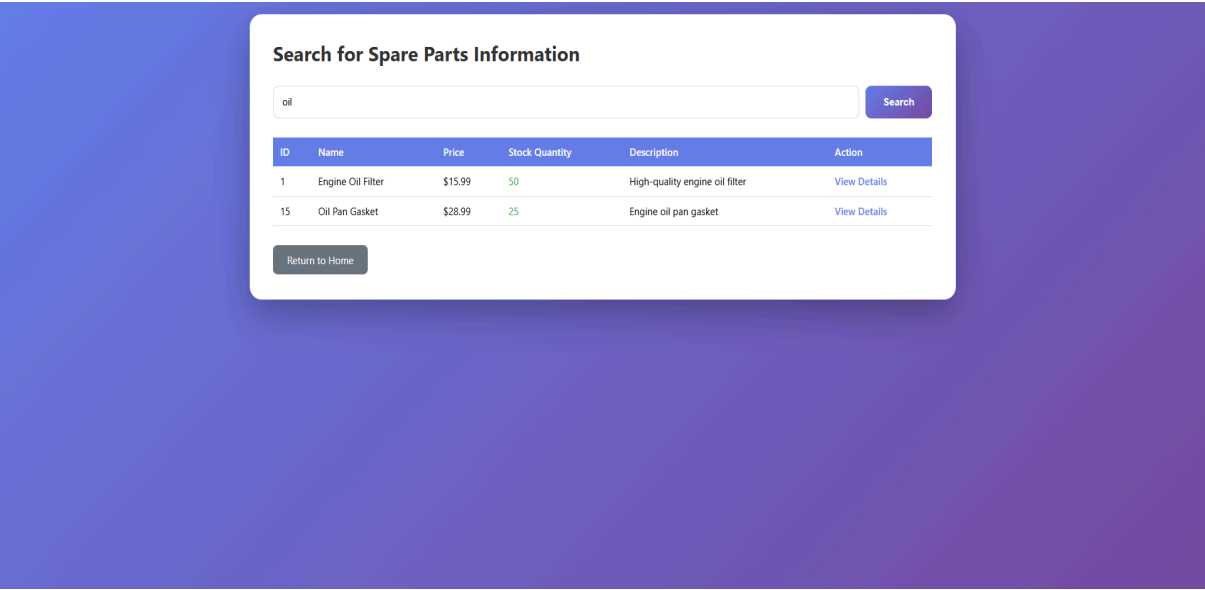


ServiceDetailsView.jsp

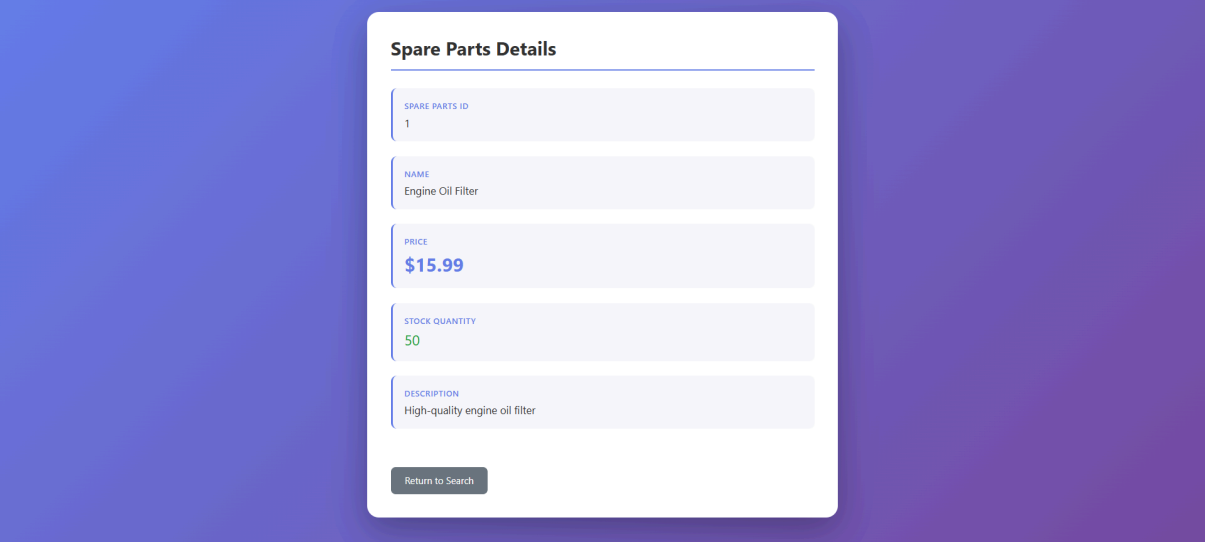


SearchForSparePartsInformationView.jsp



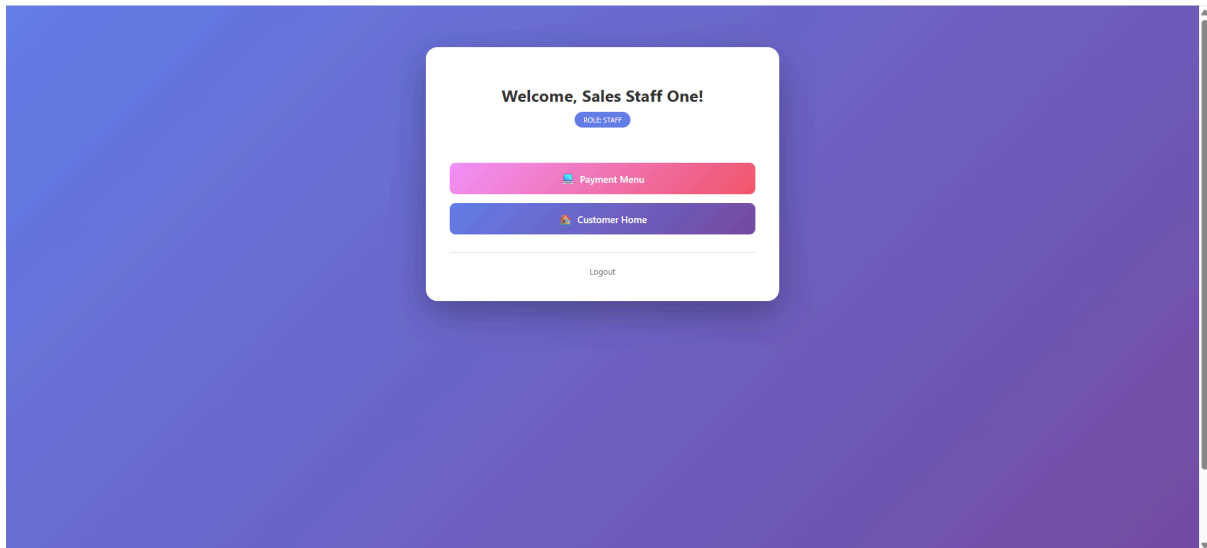


SparePartsView.jsp

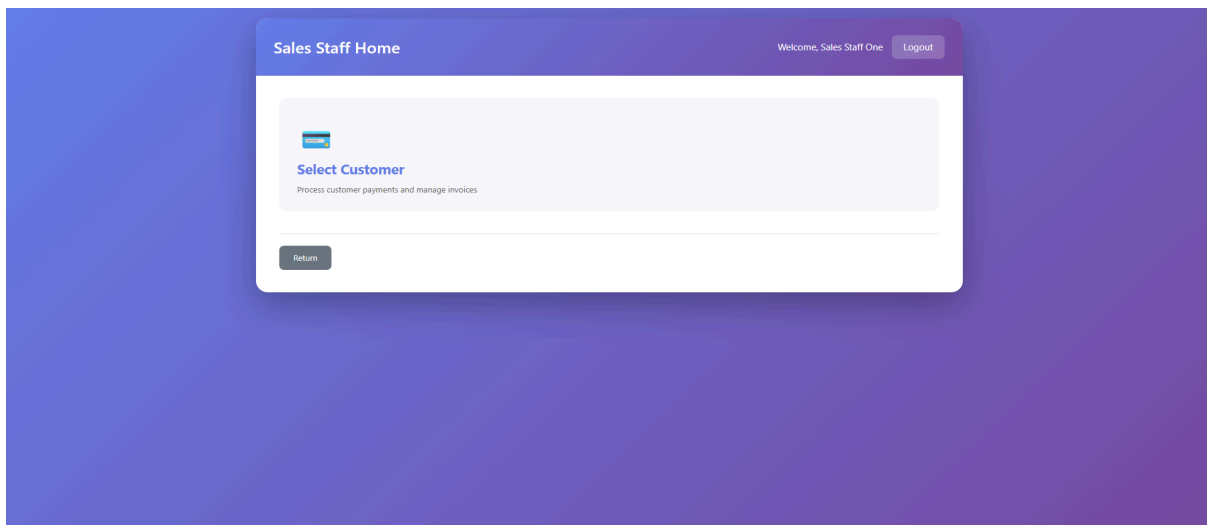


2.2 Module 2

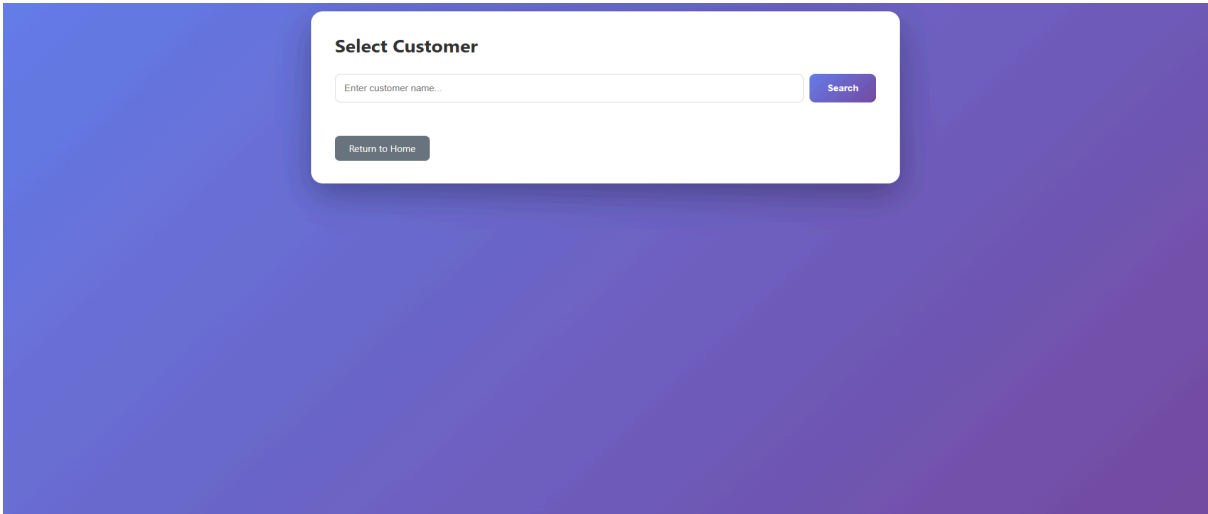
Welcome.jsp



HomeViewSaleStaff.jsp



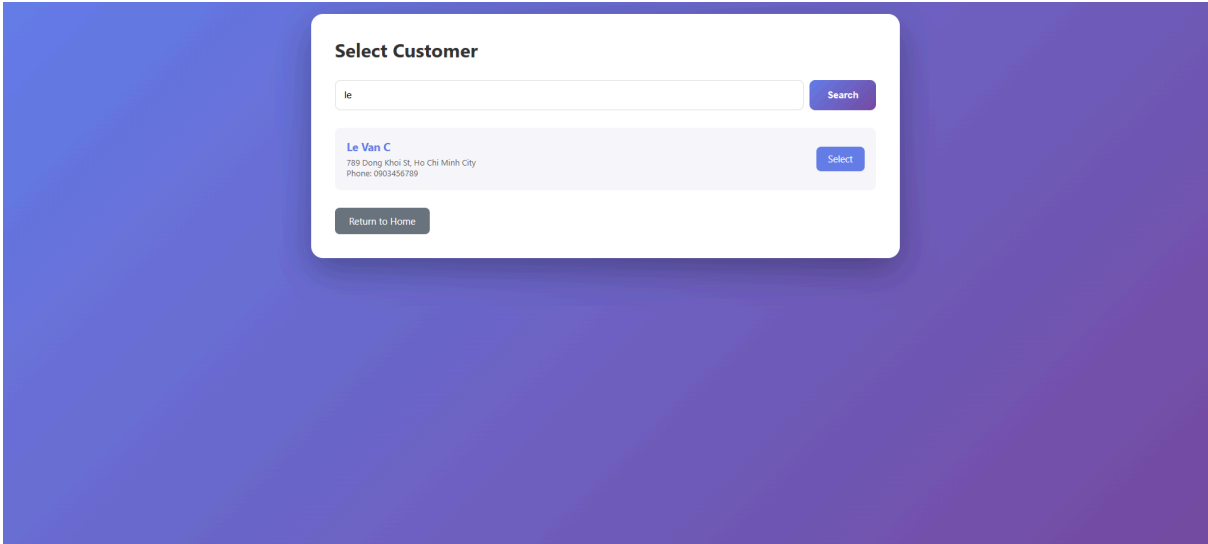
SelectCustomerView.jsp



Select Customer

Search

Return to Home



Select Customer

Search

Le Van C
789 Dong Khoi St, Ho Chi Minh City
Phone: 0903456789

Select

Return to Home

DetailedInvoicesView.jsp

Detailed Invoices

Invoice #4

Date: 2024-02-05

Status: **paid**

Total Amount: \$80.00

Vehicle Information

License Plate: 51D-22222

Brand/Model: Mazda CX-5

Year: 2022

Color: Red

Description: Mazda CX-5 2022

Services

Service ID	Service Name	Quantity	Amount
5	AC Service	1	\$80.00

Spare Parts

Spare Parts ID	Spare Parts Name	Quantity	Amount
5	Battery	1	\$120.00

Invoice #8

Date: 2024-02-25

Status: **unpaid**

Total Amount: \$200.00

Invoice #8

Date: 2024-02-25

Status: **unpaid**

Total Amount: \$200.00

Vehicle Information

License Plate: 511-77777

Brand/Model: Mitsubishi Outlander

Year: 2020

Color: Blue

Description: Mitsubishi Outlander 2020

Add Service

Search Service by Name

Enter service name

Search

Service ID

Quantity

Enter service ID

1

Add Service

Add Spare Parts

Search Spare Parts by Name

Enter spare parts name

Search

Spare Parts ID

Quantity

Enter spare parts ID

1

Add Spare Parts

Services

Service ID	Service Name	Quantity	Amount
4	Engine Tune-up	1	\$200.00
9	Car Wash	1	\$25.00

Spare Parts

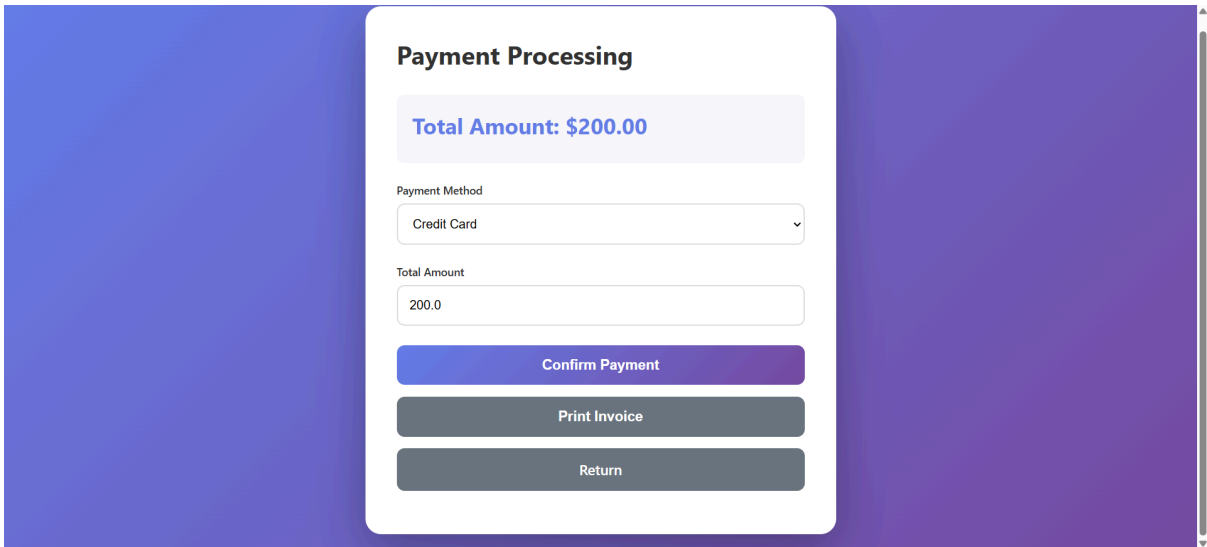
Spare Parts ID	Spare Parts Name	Quantity	Amount
17	Alternator	1	\$250.00
7	Headlight Bulb	1	\$18.99

Confirm Order

Invoice #11

Date: 2024-02-05

PaymentView.jsp



The screenshot displays a web page titled "PaymentView.jsp" with a dark blue gradient background. In the center, there is a white rounded rectangle containing a payment processing form. The form is titled "Payment Processing" in bold black text. Below the title, there is a light blue box with the text "Total Amount: \$200.00". Underneath this, there is a section labeled "Payment Method" with a dropdown menu showing "Credit Card". Below the dropdown, there is a section labeled "Total Amount" with a text input field containing "200.0". At the bottom of the form, there are three buttons: "Confirm Payment" (blue with white text), "Print Invoice" (dark gray with white text), and "Return" (dark gray with white text).

Payment Processing

Total Amount: \$200.00

Payment Method

Credit Card

Total Amount

200.0

Confirm Payment

Print Invoice

Return

3. GITHUB LINK

https://github.com/nhatgudboi/NhatB22DCCN580_PTTK